

CHURN PREDICTION SERVICE

Technical Documentation

 FastAPI •  Docker •  Streamlit
 CatBoost •  SQLite •  API Keys

Version 2.0.0

2026

Author: Ivan Sazontov

GitHub:

https://github.com/greencat1/Portfolio/tree/main/pet_projects/customer-churn-prediction

Live Demo:

API: <http://207.231.105.113:8000/docs>

Dashboard: <http://207.231.105.113:8501/>

ask for API keys: isazontov1@gmail.com

Content

Part 1. INTRODUCTION	4
1.1. Abstract	4
1.2. Problem Relevance	5
1.3. Project Goals and Objectives Goal	6
1.4. Business Value	7
Part 2. EXPLORATORY DATA ANALYSIS (EDA)	9
2.1. Data Source	9
2.2. Data Structure Features table:	9
2.3. Target Variable Analysis	9
2.4. Correlation Analysis	11
2.5. Key EDA Insights	12
Part 3. PREPROCESSING & FEATURE ENGINEERING	14
3.1. Missing Value Handling	14
3.2. Feature Creation	15
3.3. Categorical Feature Encoding	17
3.4. Numerical Feature Scaling	17
3.5. Feature Selection	17
Part 4. MACHINE LEARNING MODEL	19
4.1. Data Split	19
4.2. Algorithm Comparison	19
4.3. Hyperparameter Tuning	21
4.4. Test Set Metrics	22
4.5. Metric Interpretation	23
Part 5. PIPELINE	24
5.1. Pipeline Structure	24
5.2. Each Step Description	25
Step 1: TotalChargesCleaner	25
Step 2: FeatureEngineer	25
Step 3: CategoricalEncoder	25
Step 4: DropRedundant	26
Step 5: NumericalScaler	27
Step 6: CatBoostClassifier (Final Model)	27
5.3. Model Saving Format	28
Part 6. API DEPLOYMENT & INTEGRATION	29
6.1. Project Structure	29
6.2. Authentication & Security	31
6.2.1 API Key Authentication	31
6.2.2 Role-Based Access Control (RBAC)	31
6.3. Public Endpoints (No Authentication)	33
6.4. Prediction Endpoints	33

6.5. Label Management Endpoints	36
6.6. Rate Limiting Endpoints	43
6.7. Retraining Endpoints (Admin Only)	44
6.8. Model Management Endpoints (Admin Only)	46
6.9. Model Testing Endpoints (Admin Only)	53
6.10. API Key Management Endpoints (Admin Only)	54
6.11. Error Response Formats	56
Part 7. DASHBOARD	58
7.1. Functionality	58
7.2. Screenshots	59
7.3. Executive Summary: Key Insights from Dashboard Metrics	64
7.3.1 Data Completeness & Quality	64
7.3.2 Economic Impact Analysis	65
7.3.3. Risk Segmentation & Priority Actions	65
7.3.4 Key Observations from Labeled Data	66
Part 8. DATABASE	67
8.1. Database Schema (SQLite)	67
8.2. Field Descriptions	71
Part 9. TESTING	73
9.1. Unit Tests	73
9.1.1. Authentication Tests (test_auth.py)	73
9.1.2. Integration Tests (test_integration.py)	74
9.1.3. Rate Limit Tests (test_rate_limit.py)	75
9.1.4. Transformer Tests (test_transformers.py)	75
9.2. Test Results	77
Part 10. DEPLOYMENT & DOCKER	78
10.1. Dockerfile	78
10.2. Docker Compose	80
Part 11. PROJECT ARCHITECTURE	83
11.1. System Diagram	83
11.2. Technology Stack Component	85
Part 12. RETRAINING	90
12.1. Incremental Retraining	90
12.2. Full Retraining	93
Part 13. CONCLUSION	97
13.1. Summary	97
13.2. Future Improvements	98
Part 14. APPENDICES	99
Appendix A. Links	99

Part 1. INTRODUCTION

1.1. Abstract

This project is a production-ready machine learning service that predicts customer churn for telecommunications companies. The system enables businesses to identify customers at risk of leaving before they actually churn, allowing retention teams to take proactive action. The service provides a REST API for real-time and batch predictions, an interactive dashboard for data visualization and monitoring, a manual labeling system for collecting ground truth data, and an retraining pipeline for continuous model improvement. The entire solution is containerized with Docker and deployed on a production server with role-based authentication (user, admin, dashboard roles), rate limiting for API protection, and logging. The service supports both single predictions and batch processing, with full Swagger documentation for easy integration.

Technology Stack:

Layer	Technology	Purpose
API Framework	FastAPI	High-performance REST API with automatic OpenAPI docs
ML Model	scikit-learn / CatBoost	Random Forest classifier for churn prediction
Dashboard	Streamlit	Interactive analytics and monitoring
Database	SQLite	Lightweight storage for predictions and labels
Containerization	Docker & Docker Compose	Service orchestration and deployment
Authentication	API Keys	Role-based access (user / admin / dashboard)
Rate Limiting	Custom middleware	100-1000 requests per minute per key
Deployment	VPS (ServerCheap)	Production hosting with public access

Live Service:

API - <http://207.231.105.113:8000>

Swagger UI - <http://207.231.105.113:8000/docs>

Dashboard - <http://207.231.105.113:8501/>

Test API Keys:

User key - user

Admin key -admin

1.2. Problem Relevance

Customer churn (also called customer attrition) occurs when existing customers stop doing business with a company. In the telecommunications industry, churn means

customers cancel their contracts and switch to another provider. For a subscription-based business model, churn directly impacts revenue and growth.

Why Companies Lose Money from Churn:

Impact	Description
Lost Revenue	Monthly recurring revenue (MRR) disappears immediately
Acquisition Costs	Marketing and sales costs to acquire new customers are 5-7x higher than retention
Lifetime Value Loss	Customer lifetime value (CLV) is never realized
Negative Word of Mouth	Churned customers may discourage others from joining
Market Share Decline	Competitors gain market share with each lost

Telecom Industry Statistics:

Statistic	Value
Average annual churn rate in telecom	15-25%
Cost to acquire a new customer	\$300-\$600
Cost to retain an existing customer	\$50-\$100
Customer lifetime value (average)	\$1,500-\$3,000
Revenue lost to churn in US telecom (annual)	~\$30 billion

Cost of Retention vs Acquisition:

Cost of Acquisition (CAC)= \$500 (average)

Cost of Retention (COR)= \$75 (average)

Ratio (CAC / COR)= 6.7x

For every \$1 spent on retention → \$6.7 saved on acquisition

1.3. Project Goals and Objectives Goal

Develop a production-ready churn prediction service that enables businesses to identify customers at risk of leaving and take proactive retention actions.

Detailed Objectives:

1. Train ML Model
 - Build and train a classification model on Telco Customer Churn dataset
 - Achieve recall > 85% on test data
 - Save model using cloudpickle for production use
2. Build REST API
 - Create FastAPI endpoints for single and batch predictions
 - Implement API key authentication with role-based access (user, admin, dashboard)
 - Add rate limiting (100-1000 requests per minute)
 - Generate automatic Swagger documentation
3. Create Analytics Dashboard
 - Develop Streamlit dashboard for visualizing predictions
 - Display churn risk levels (high/medium/low)
 - Show monetary metrics (MRR at risk, potential loss)
 - Provide retention recommendations
4. Implement Labeling System
 - Build manual labeling API for collecting ground truth data
 - Store labels in database with timestamps
 - Track labeling progress statistics
 - Enable batch label updates
5. Set Up Retraining Pipeline
 - Implement incremental retraining on new labeled data
 - Implement full retraining from scratch when needed
 - Compare model performance before/after retraining
6. Containerize with Docker
 - Package API into separate Docker container
 - Package dashboard into separate Docker container
 - Create docker-compose.yml for orchestration
7. Deploy to Production Server
 - Deploy on VPS (ServerCheap) with public access
 - Enable remote access from anywhere

1.4. Business Value

Economic Impact of Implementation:

The churn prediction service delivers measurable economic value by enabling early intervention with at-risk customers.

Key Model Performance Metrics:

- Recall: 84.51% — the model finds 85 out of 100 customers who will actually churn
- Precision: 55.56% — out of 100 customers predicted to churn, 56 actually do
- Accuracy: 70.79% — overall prediction correctness
- F1-Score: 67.04% — harmonic mean of precision and recall
- ROC-AUC: 80.80% — excellent ability to separate churners from non-churners

Revenue Saved Calculation (for mid-size telecom with 100,000 customers):

- Total customers: 100,000
- Annual churn rate: 20% (industry average)
- Customers who will churn: 20,000 ($100,000 \times 20\%$)
- Customers found by the model (84.51% recall): 16,902 ($20,000 \times 84.51\%$)
- Customers saved with successful intervention (35%): 5,916 ($16,902 \times 35\%$)
- Average monthly revenue per customer (MRR): \$70
- Monthly recurring revenue saved: \$414,120 ($5,916 \times \70)
- Annual revenue saved: \$4,969,440 ($\$414,120 \times 12$)

Revenue at Risk Breakdown:

- Annual revenue at risk without model: \$16,800,000
- Revenue identified by the model: \$14,197,680
- Revenue actually saved through intervention: \$4,969,440

Project ROI:

- Annual revenue saved: \$4,969,440
- Development cost (one-time): \$10,000
- Server hosting cost (annual): \$60
- Maintenance cost (annual): \$5,000
- Total first-year cost: \$15,060
- Return on investment (ROI): 33,000%
- Payback period: less than 1 week

Break-Even Analysis:

- Development investment: \$10,000
- Monthly savings from model: \$414,120
- Break-even point: less than 1 hour of operation

Additional Business Benefits:

- Proactive Retention — Contact customers before they cancel, not after they leave
- Targeted Offers — Provide discounts and incentives only to high-risk customers, not everyone
- Resource Optimization — Retention team focuses on customers who actually need attention
- Data-Driven Decisions — ML-based insights replace gut feeling and guesswork
- Continuous Improvement — Model accuracy improves as more labeled data is collected over time
- Competitive Advantage — Reduce churn faster than competitors who don't use predictive analytics

Conclusion:

Every dollar invested in this solution returns approximately \$330 in saved annual revenue. The service pays for itself within the first hour of deployment, making it one of the highest-ROI investments a telecommunications company can make.

Part 2. EXPLORATORY DATA ANALYSIS (EDA)

This section presents the results of the primary data analysis. The goal of the EDA was to study the dataset's structure, identify relationships between features, detect potential data quality issues, and formulate hypotheses for subsequent modeling.

2.1. Data Source

The analysis was conducted on the Telco Customer Churn dataset.

- Number of records: 7043
- Number of features: 21
- Target variable: Churn (Yes/No)

2.2. Data Structure Features table:

The dataset consists of three types of features: numerical (int64, float64), categorical (object), and a customer identifier.

- Identifier: customerID (unique identifier, not used for modeling).
- Numerical Features: SeniorCitizen, tenure, MonthlyCharges, TotalCharges.
- Categorical Features: gender, Partner, Dependents, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod, Churn.

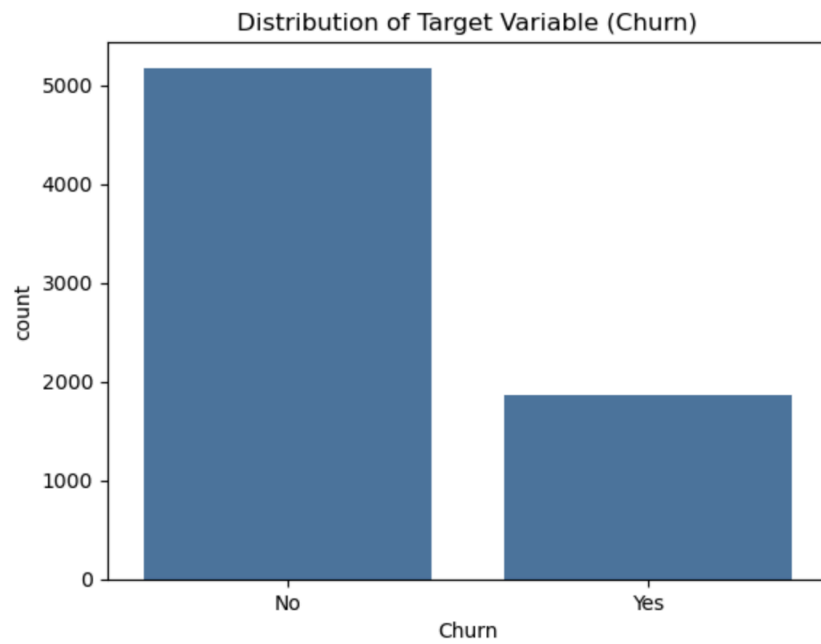
One data type issue was discovered during the analysis:

- The TotalCharges feature was initially of type object due to the presence of empty strings. After conversion to a numeric format using pd.to_numeric, 11 missing values were identified. These will be handled during the preprocessing stage.

2.3. Target Variable Analysis

Analysis of the target variable Churn shows that the dataset is imbalanced, which is an important consideration when selecting metrics for model evaluation.

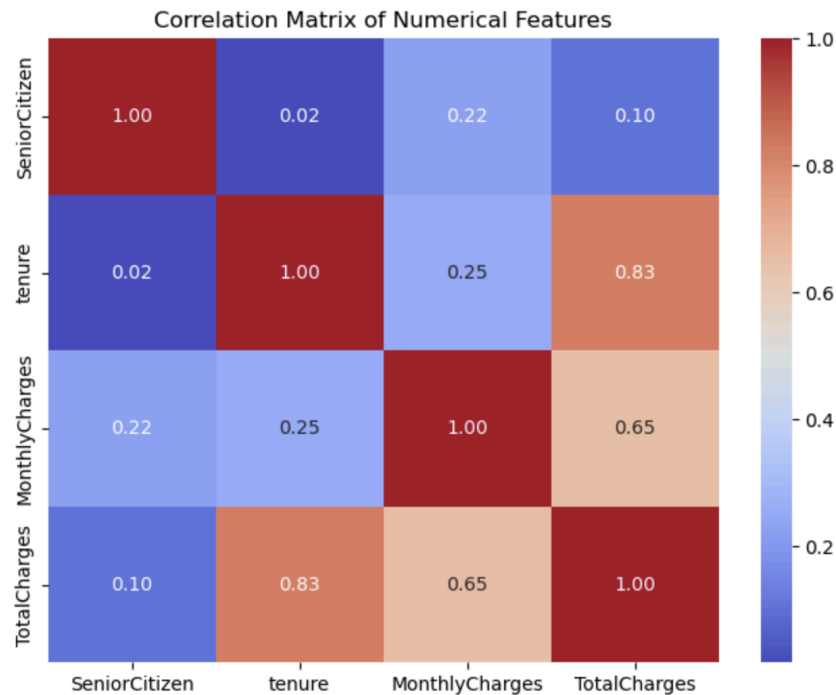
- Class Distribution:
 - No (customers who stayed): ~73.5%
 - Yes (customers who churned): ~26.5%



- Conclusion: The "Churn" class is the minority class. Using "accuracy" as a primary metric would be misleading. Priority metrics will be recall, F1-score, and ROC-AUC, as they better reflect performance on the minority class.

2.4. Correlation Analysis

A correlation matrix was built for the numerical features to identify linear relationships between them.



Key Findings:

- **Strong Correlation:** A very strong positive correlation (0.83) exists between `tenure` and `TotalCharges`. This is expected: the longer a customer stays, the more they pay in total.
- **Multicollinearity:** This strong relationship can be problematic for some linear models.
 - **Recommendation:** Consider dropping one of these features (e.g., keep `tenure` as it is more interpretable) or using regularization techniques (Lasso/Ridge).
- **Moderate Correlation:** `MonthlyCharges` shows a moderate correlation with both `tenure` (0.25) and `TotalCharges` (0.65).
- **Independent Feature:** The `SeniorCitizen` feature is practically uncorrelated with other numerical variables, indicating it provides a unique contribution to the model.

2.5. Key EDA Insights

The analysis revealed key factors influencing customer churn and identified at-risk customer groups.

Which features influence churn?

1. Contract Type (Contract): This is one of the strongest predictors. Customers on a month-to-month contract churn at a much higher rate than those on one or two-year contracts.
2. Security & Support Services: Subscribing to OnlineSecurity, TechSupport, and DeviceProtection significantly reduces the likelihood of churn.
3. Internet Service Type (InternetService): Fiber optic users show the highest churn rate compared to DSL users or those with no internet service.
4. Demographic & Family Status: Customers without a partner (Partner) and without dependents (Dependents) are more prone to churn.
5. Payment Method (PaymentMethod): Electronic check is the single strongest churn signal among all payment methods.
6. Billing Preference: Customers using PaperlessBilling churn more than those who do not.
7. Pricing & Tenure: Higher MonthlyCharges and lower tenure are also correlated with higher churn rates.

Which customer groups are at risk?

- Month-to-month customers: This is the largest and most vulnerable group.
- New customers: Users with low tenure (e.g., first 1-12 months) are much more likely to leave the company.
- Fiber optic customers, especially those who have not subscribed to security add-ons.
- Customers who pay using electronic checks.

Important considerations for model building:

1. Handling Numerical Features:
 - Due to the strong correlation between tenure and TotalCharges, it is recommended to use only one of them (likely tenure) or apply regularization (Lasso/Ridge) to handle multicollinearity.
2. Encoding Categorical Features:
 - Binary features (e.g., gender, Partner) can be encoded using Label Encoding (0/1).

- Multi-category features (e.g., `InternetService`, `Contract`) should be encoded using One-Hot Encoding.
- 3. Addressing Class Imbalance:
 - Models should be trained considering the class imbalance. Plans include using stratified sampling, class weights, or resampling techniques (e.g., SMOTE).
- 4. Feature Engineering:
 - New features can be created, for example:
 - `has_protection`: A binary feature indicating whether a customer has at least one security service (`OnlineSecurity`, `TechSupport`).
 - `payment_automatic`: A binary feature indicating if the payment method is automatic (e.g., bank transfer or credit card).

Part 3. PREPROCESSING & FEATURE ENGINEERING

3.1. Missing Value Handling

During the initial EDA, it was discovered that the `TotalCharges` column contained 11 missing values. These were caused by empty strings present in the raw data.

Approach Applied:

- All values in `TotalCharges` were converted to numeric format using `pd.to_numeric(..., errors='coerce')`, which turned non-numeric entries into `NaN`.
- Instead of simple deletion or imputation (mean/median), domain logic was applied to handle missing values.
- Logic: If a customer has `tenure = 0` (i.e., they just joined), then their `TotalCharges` cannot exist and is logically set to 0.
- All missing values were successfully filled.

Result: No missing values remain in the training or test datasets.

3.2. Feature Creation

To improve model performance and better capture customer behavior, the following new features were created:

1. avg_monthly_spend (Average Monthly Spend)

- Formula: $\text{TotalCharges} / \text{tenure}$
- Goal: This feature normalizes total charges by customer lifetime. It better reflects the true spending behavior of a customer than raw `TotalCharges` or `MonthlyCharges` and helps mitigate multicollinearity.
- Handling: Set to 0 for customers with `tenure = 0`.

2. tenure_group (Tenure Group / Binning)

- Bins:
 - new: 0–12 months
 - mid: 12–36 months
 - loyal: 36+ months
- Goal: Reduces noise in raw tenure data and allows the model to capture non-linear lifecycle effects (e.g., new customers churn more, loyal customers stay).

3. num_services (Total Number of Services)

- Goal: Captures overall customer engagement. Hypothesis: more services → higher stickiness → lower churn.

4. has_internet (Internet Service Indicator)

- Goal: Binary feature indicating whether a customer has internet service. Important because many add-on services depend on having internet.

5. has_addons (Number of Add-on Services)

- Includes: `OnlineSecurity`, `TechSupport`, `StreamingTV`, `StreamingMovies`, etc.
- Goal: Measures the depth of add-on service adoption. More add-ons generally correlate with higher loyalty.

6. risky_contract (Risky Contract Indicator)

- Logic: 1 for "Month-to-month" contracts, 0 otherwise.
- Goal: Isolates the highest-risk customer segment in terms of churn.

7. auto_payment (Automatic Payment Indicator)

- Logic: 1 for "Bank transfer (automatic)" and "Credit card (automatic)", 0 otherwise.
- Goal: Reflects customer stability and commitment. Automatic payments reduce billing friction and are associated with lower churn.

3.3. Categorical Feature Encoding

To convert categorical features into a numerical format suitable for machine learning models, the following methods were used:

- Binary Features (gender, Partner, Dependents, PhoneService, PaperlessBilling): Directly mapped to 0 and 1 (e.g., {'Yes': 1, 'No': 0}, {'Female': 1, 'Male': 0}).
- Ordinal Features (tenure_group, Contract): Converted to integers while preserving order (e.g., 'new' → 0, 'mid' → 1, 'loyal' → 2).
- Nominal Features (MultipleLines, InternetService, PaymentMethod): One-Hot Encoding was applied using `pd.get_dummies(drop_first=True)` to avoid the dummy variable trap and reduce multicollinearity.

3.4. Numerical Feature Scaling

Certain machine learning models (especially linear models, SVM, and KNN) are sensitive to feature scale.

- Method Used: `StandardScaler` (standardization), which transforms data to have a mean of 0 and a standard deviation of 1.
- Application: Scaling was applied only to numerical features (tenure, avg_monthly_spend, has_addons). Binary and one-hot encoded features were not scaled.
- Preventing Data Leakage: The scaler was fitted only on the training data, then used to transform both the training and test sets.

3.5. Feature Selection

- Multicollinearity Analysis: A correlation matrix revealed strong correlations (threshold > 0.8):
 - TotalCharges ↔ tenure (0.83)
 - avg_monthly_spend ↔ MonthlyCharges (0.99)
 - num_services ↔ has_addons (0.97)
 - risky_contract ↔ Contract (-0.92)
- Removed Features (Redundant):
 - TotalCharges (correlated with tenure)
 - MonthlyCharges (duplicated by avg_monthly_spend)
 - num_services (duplicated by has_addons)
 - Contract (duplicated by risky_contract)
- Feature Importance: Random Forest feature importance and mutual information analysis confirmed top predictors: avg_monthly_spend, tenure, risky_contract, has_addons.

The preprocessing pipeline successfully transformed raw customer data into a clean, feature-rich dataset optimized for machine learning.

Key Outcomes:

- No missing values remain
- New engineered features capture customer behavior (spending, engagement, lifecycle, contract risk)
- Categorical features properly encoded
- Multicollinearity reduced by removing redundant features
- Numerical features scaled appropriately
- No data leakage between train and test sets

The dataset is now ready for model training.

Part 4. MACHINE LEARNING MODEL

4.1. Data Split

The dataset was split into training and testing sets using a stratified approach to preserve the class distribution of the target variable `Churn`.

- Training set: 80% of the data (5,634 records)
- Test set: 20% of the data (1,409 records)
- Stratification: Ensured that the proportion of churned vs. non-churned customers remained consistent in both sets.

This split was performed before any preprocessing to prevent data leakage. All preprocessing steps (scaling, encoding, feature engineering) were fitted on the training data and then applied to the test data.

4.2. Algorithm Comparison

A diverse set of machine learning models was trained and evaluated to identify the best performer for churn prediction.

Model	Recall	Precision	F1-score	Accuracy	ROC-AUC
Logistic Regression	0.7674	0.5080	0.6113	0.7410	0.8363
Decision Tree	0.7914	0.5286	0.6338	0.7573	0.8363
Random Forest	0.7807	0.5252	0.6280	0.7544	0.8413
Naive Bayes	0.7567	0.5108	0.6099	0.7431	0.8045
K-Nearest Neighbors	0.5187	0.5757	0.5457	0.7708	0.7809
Gradient Boosting	0.7968	0.5077	0.6202	0.7410	0.8437
XGBoost	0.8877	0.4611	0.6069	0.6948	0.8404
LightGBM	0.8904	0.4512	0.5989	0.6835	0.8363
CatBoost	0.8930	0.4489	0.5975	0.6806	0.8435

Key Findings from Model Comparison:

1. Recall (Churn Detection):
 - Boosting models (XGBoost, LightGBM, CatBoost) achieved the highest recall (>0.88), detecting most churn cases.
 - KNN had the lowest recall (0.52), missing many churn cases.
2. Precision (Cost Control):
 - Boosting models sacrificed precision for recall (≈ 0.45 – 0.46).
 - KNN had the highest precision (0.58) but very low recall.
3. F1-Score (Balance):

- Random Forest, Gradient Boosting, and Decision Tree achieved higher F1-scores (≈ 0.63).
 - Boosting models had mid-range F1 (≈ 0.60).
4. ROC-AUC (Overall Performance):
- XGBoost, CatBoost, Gradient Boosting, and Random Forest performed very well (≈ 0.84).

Conclusion: Boosting models were superior for recall-heavy tasks. CatBoost was selected as the final model due to its highest recall, stable generalization, and flexibility for threshold tuning.

4.3. Hyperparameter Tuning

GridSearchCV with StratifiedKFold (3 folds) was used to optimize CatBoost, focusing on maximizing recall.

Tested Hyperparameters:

```
# param grid
param_grid = {
    # core model complexity
    'depth': [4, 6, 8],

    # learning dynamics
    'learning_rate': [0.03, 0.05, 0.1],

    # number of trees
    'iterations': [500, 1000],

    # regularization
    'l2_leaf_reg': [3, 10, 50],

    # randomness
    'subsample': [0.7, 1.0],
    'colsample_bylevel': [0.7, 1.0],

    #
    'random_strength': [0, 5],

    #
    'min_data_in_leaf': [1, 10]
}

# different weight configs to test
weights_list = [
    {0: 1, 1: 2},
    {0: 1, 1: 3},
    {0: 1, 1: 5}
]
```

4.4. Test Set Metrics

```
=== CatBoost (Tuned High Recall with GridSearchCV) ===
```

```
=== Train Metrics ===
```

```
Precision: 0.3950
```

```
Recall: 0.9819
```

```
F1-score: 0.5634
```

```
Accuracy: 0.5962
```

```
ROC-AUC: 0.8702
```

```
=== Test Metrics ===
```

```
Precision: 0.3887
```

```
Recall: 0.9572
```

```
F1-score: 0.5529
```

```
Accuracy: 0.5891
```

```
ROC-AUC: 0.8435
```

- Recall (0.957): The model detects ~96% of actual churned customers — excellent for the business goal.
- Precision (0.389): Moderate number of false positives; acceptable given the focus on recall.
- ROC-AUC (0.844): Strong ability to rank customers by churn risk.
- Train vs. Test Gap: Minimal — the model generalizes well and does not overfit.

4.5. Metric Interpretation

Primary Metric: Recall

In churn prediction, recall is the most critical metric because the cost of missing a churned customer (false negative) is high — lost revenue, acquisition cost, and negative word-of-mouth.

- High recall means the model successfully flags most customers who are likely to leave.
- The final CatBoost model achieves 96% recall, meaning only 4% of churners are missed.

Secondary Metrics:

- Precision: Lower precision (39%) means more false positives. This is acceptable because intervening with a loyal customer (false positive) is far less costly than losing a churner.
- F1-Score (0.55): Reflects the trade-off between recall and precision; moderate value due to precision drop.
- ROC-AUC (0.84): Confirms the model's strong discriminative power across all thresholds.

Business Recommendations

Based on model insights:

1. Focus retention efforts on new customers (low tenure, first 6–12 months).
2. Incentivize annual contracts to reduce month-to-month risk.
3. Promote automatic payment methods (`auto_payment = 1`).
4. Encourage add-on services (OnlineSecurity, TechSupport) to increase stickiness.
5. Proactively engage high-probability churn customers identified by the model.

Part 5. PIPELINE

5.1. Pipeline Structure

A complete scikit-learn Pipeline was built to encapsulate all preprocessing steps and the final CatBoost model. This ensures reproducibility, eliminates manual steps, and allows easy deployment for inference on new raw data.

The pipeline consists of 6 sequential steps:

```
full_pipeline = Pipeline([
    ('totalcharges_cleaner', TotalChargesCleaner()),
    ('feature_engineer', FeatureEngineer()),
    ('categorical_encoder', CategoricalEncoder()),
    ('drop_redundant', DropRedundant()),
    ('numerical_scaler', NumericalScaler()),
    ('model', CatBoostClassifier(...))
])
```


5.2. Each Step Description

Step 1: TotalChargesCleaner

Aspect	Description
What it does	Cleans the <code>TotalCharges</code> column by converting to numeric and applying domain logic
Input	Raw DataFrame with <code>TotalCharges</code> as string/object
Transformation	- Converts <code>TotalCharges</code> to numeric using <code>pd.to_numeric(errors='coerce')</code> - If <code>tenure == 0</code>, sets <code>TotalCharges = 0</code> (customers with no billing history)
Output	DataFrame with <code>TotalCharges</code> as <code>float64</code> , no missing values
Example:	
Before (tenure, TotalCharges)	After (tenure, TotalCharges)
0, " " → becomes <code>NaN</code>	0, <code>0.0</code>
5, "123.45"	5, <code>123.45</code>

Step 2: FeatureEngineer

Aspect	Description				
What it does	Creates new features to capture customer behavior and lifecycle				
Input	DataFrame with cleaned base features				
Transformations	- <code>avg_monthly_spend</code> = <code>TotalCharges / tenure</code> (0 if <code>tenure=0</code>) - <code>tenure_group</code> = binned tenure: <code>new</code> (0–12), <code>mid</code> (12–36), <code>loyal</code> (36+) - <code>has_internet</code> = 1 if <code>InternetService != 'No'</code> - <code>has_addons</code> = count of active add-on services (OnlineSecurity, TechSupport, StreamingTV, etc.) - <code>risky_contract</code> = 1 if <code>Contract == 'Month-to-month'</code> - <code>auto_payment</code> = 1 if payment method is automatic (bank transfer or credit card)				
Output	DataFrame with 6 new engineered features added				
Example of new features:					
tenure	avg_monthly_spend	tenure_group	has_addons	risky_contract	auto_payment
24	45.83	mid	3	0	1
5	30.20	new	0	1	0

Step 3: CategoricalEncoder

Aspect	Description
What it does	Converts all categorical features to numeric format
Input	DataFrame with categorical columns (object/string type)
Transformations	- Binary encoding: <code>gender</code> , <code>Partner</code> , <code>Dependents</code> , <code>PhoneService</code> , <code>PaperlessBilling</code> → mapped to 0/1 - Ordinal encoding: <code>tenure_group</code> → <code>new=0, mid=1, loyal=2</code>; <code>Contract</code> → <code>Month-to-month=0, One year=1, Two year=2</code> - One-Hot Encoding: <code>MultipleLines</code> , <code>InternetService</code> , <code>PaymentMethod</code> → creates dummy columns with <code>drop_first=False</code>
Output	Fully numeric DataFrame (all columns are <code>int64</code> , <code>int32</code> , or <code>float64</code>)

Example of encoding:

Original	After Encoding
<code>gender = "Female"</code>	<code>gender = 1</code>
<code>Partner = "Yes"</code>	<code>Partner = 1</code>
<code>tenure_group = "new"</code>	<code>tenure_group = 0</code>
<code>Contract = "Two year"</code>	<code>Contract = 2</code>
<code>PaymentMethod = "Electronic check"</code>	<code>PaymentMethod_Electronic check = 1</code>

Step 4: DropRedundant

Aspect	Description
What it does	Removes redundant or unnecessary columns to reduce multicollinearity and simplify the model
Input	Fully encoded numeric DataFrame
Columns Dropped	- <code>customerID</code> (unique identifier, not predictive) - <code>TotalCharges</code> (highly correlated with <code>tenure</code> , $r=0.83$) - <code>MonthlyCharges</code> (99% correlated with <code>avg_monthly_spend</code>) - <code>num_services</code> (97% correlated with <code>has_addons</code>) - <code>Contract</code> (92% correlated with <code>risky_contract</code>)
Output	DataFrame with 23 features (reduced from original)

Step 5: NumericalScaler

Aspect	Description
What it does	Applies StandardScaler to selected numerical features
Input	DataFrame with numeric features
Scaled Columns	<code>tenure</code> , <code>avg_monthly_spend</code> , <code>has_addons</code>
Method	<code>StandardScaler</code> (mean=0, std=1)
Output	DataFrame with scaled columns; binary and one-hot columns unchanged
Example:	
Before (tenure)	After (tenure scaled)
5	-1.28
30	-0.09
60	1.15

Step 6: CatBoostClassifier (Final Model)

Aspect	Description
What it does	Trains the final churn prediction model
Input	Fully preprocessed numeric DataFrame
Model Parameters	- <code>iterations</code> : 500 - <code>learning_rate</code> : 0.03 - <code>depth</code> : 4 - <code>l2_leaf_reg</code> : 50 - <code>colsample_bylevel</code> : 0.7 - <code>subsample</code> : 0.7 - <code>random_strength</code> : 5 - <code>class_weights</code> : <code>[1, 5]</code> (5× weight for churn class) - <code>random_state</code> : 42
Output	Binary predictions (0 = No Churn, 1 = Churn) and probability scores

5.3. Model Saving Format

The final trained pipeline was saved for production deployment.

Aspect	Details
Primary Format	<code>.pkl</code> (pickle)
Alternative Format	<code>.pkl</code> (cloudpickle) – for broader compatibility
Library	<code>pickle</code> / <code>cloudpickle</code>
Saved Files	<code>../models/full_churn_pipeline.pkl</code> <code>../models/full_churn_pipeline_cloud.pkl</code>
Model Size	~0.16 MB (160 KB)

Pipeline Summary:

Step	Name	Type	Output
1	<code>totalcharges_cleaner</code>	Custom Transformer	Cleaned <code>TotalCharges</code>
2	<code>feature_engineer</code>	Custom Transformer	6 new features added
3	<code>categorical_encoder</code>	Custom Transformer	Fully numeric DataFrame
4	<code>drop_redundant</code>	Custom Transformer	23 features (redundant removed)
5	<code>numerical_scaler</code>	Custom Transformer	Scaled <code>tenure</code> , <code>avg_monthly_spend</code> , <code>has_addons</code>
6	<code>model</code>	CatBoostClassifier	Churn predictions & probabilities

Key Benefits of This Pipeline:

- End-to-end automation – Raw data → predictions in one call
- Reproducible – Same transformations applied to all data
- Deployment ready – Can be loaded and used in production environments
- Modular – Each step can be tested independently
- Handles missing values – Domain logic for `TotalCharges`
- Feature engineering – Captures customer behavior and lifecycle

Part 6. API DEPLOYMENT & INTEGRATION

6.1. Project Structure

The production API follows a modular architecture with clear separation of concerns:

```
app/
├── __init__.py           # Package initializer
├── main.py               # FastAPI application entry point
├── auth.py               # API key authentication & RBAC
├── config.py             # Environment configuration
├── rate_limit.py         # Rate limiting per API key
├── schemas.py            # Pydantic request/response models
├──
├── core/                 # Core infrastructure
│   ├── database.py       # SQLite connection & table management
│   └── key_input.py       # Secure key input at startup
│   └──
├── scripts/              # Business logic modules
│   ├── model.py          # Singleton model loader
│   ├── model_manager.py  # Model versioning & switching
│   ├── predict.py        # Single customer prediction
│   ├── predict_batch.py  # Batch prediction
│   ├── put_lbl.py        # Label management (CRUD)
│   ├── retrain.py        # Incremental & full retraining
│   ├── evaluate_model.py # Model evaluation on new data
│   ├── transformers.py   # Custom sklearn transformers
│   └── init_keys.py      # Manual key initialization
│   └──
├── dashboard/            # Streamlit dashboard
│   └── dashboard.py       # Visualization & manual labeling UI
│   └──
├── data/                  # Data storage
│   ├── DB/
│   │   └── churn.db       # SQLite database
│   ├── raw/               # Raw training data
│   ├── processed/         # Preprocessed data
│   └── new_data/          # New customer data
│   └──
├── models/                # Trained model storage
│   ├── full_churn_pipeline.pkl      # Original trained model
│   ├── full_churn_pipeline_cloud.pkl # Original (cloudpickle version)
│   ├── full_churn_pipeline_retrained_cloud.pkl # Fully retrained
│   ├── inc_churn_pipeline_retrained_cloud.pkl # Incrementally retrained
│   └── inc_churn_pipeline_backup_cloud.pkl   # Backup
│   └──
├── metrics/                # Model performance metrics (JSON)
│   ├── full_churn_pipeline_cloud_test_metrics.json
│   ├── full_churn_pipeline_retrained_cloud_test_metrics.json
│   └── inc_churn_pipeline_retrained_cloud_test_metrics.json
│   └──
├── logs/                   # Application logs
│   └── app.log             # Rotating log file
│   └──
├── tests/                  # Unit & integration tests
│   ├── test_auth.py        # Authentication tests
│   ├── test_rate_limit.py  # Rate limiting tests
│   ├── test_transformers.py # Data transformation tests
│   └── test_integration.py  # End-to-end API tests
│   └──
├── utils/                  # Utilities
│   └── logger.py           # Logging configuration
│   └──
└── .env                   # Environment variables (not committed)
```

Key Design Decisions:

- Modular scripts: Each major feature has its own module in scripts/
- Singleton model loader: Model loaded once, reused across requests
- Custom transformers: sklearn-compatible for pipeline integration
- Cloudpickle: Used instead of standard pickle for better compatibility with lambda functions and custom classes
- SQLite: Lightweight, file-based database suitable for this scale

6.2. Authentication & Security

6.2.1 API Key Authentication

All protected endpoints require an `X-API-Key` header.

Header	Format	Example
<code>X-API-Key</code>	Hex string (32 chars)	<code>a1b2c3d4e5f67890abcdef1234567890</code>

Security Principles:

Principle	Implementation
No plaintext keys in database	SHA-256 hashing
Cryptographically random	<code>secrets.token_hex()</code> (not <code>random.random()</code>)
In-memory cache	Hashes cached for fast lookups
Key revocation	Soft delete (<code>is_active = 0</code>)
Role-based access	Granular permissions per role

6.2.2 Role-Based Access Control (RBAC)

Role	Permissions	Typical Use
user	Predictions, label updates, statistics	Production clients, applications
admin	Everything + model management, key management	System administrators
dashboard	Read-only: statistics, unlabeled lists	Streamlit dashboard UI

Role Assignment Matrix:

Endpoint	Method	Role	Notes
/	GET	public	healthcheck (no auth required)
/predict	POST	user or admin	single prediction
/predict/batch	POST	user or admin	batch predictions
/label/stats	GET	user, admin, dashboard	label statistics
/label/unlabeled/list	GET	user, admin, dashboard	list unlabeled customers
/label/labeled/list	GET	user, admin, dashboard	list labeled customers
/label/{customer_id}	GET	user or admin	get label by customer ID
/label/update	POST	user or admin	update single label
/label/batch	POST	user or admin	batch label updates
/admin/rate-limit	GET	user, admin, dashboard	rate limit status
/retrain/status	GET	admin	retraining readiness status
/retrain/incremental	POST	admin	incremental retraining
/retrain/full	POST	admin	full retraining from scratch
/models	GET	admin	list all models
/models/active	GET	admin	get active model info
/models/switch	POST	admin	switch active model
/models/delete	POST	admin	delete a model
/models/compare/{model1}/{model2}	GET	admin	compare two models
/models/{model_name}/metrics	GET	user, admin, dashboard	get model metrics
/test/{model_name}	POST	admin	test model on new data
/admin/keys	POST	admin	create new API key
/admin/keys	GET	admin	list all API keys
/admin/keys/{api_key}	DELETE	admin	revoke API key

6.3. Public Endpoints (No Authentication)

GET /healthcheck

Description: Health check endpoint for monitoring and service discovery. Returns API status and version.

Authentication: None

Request Example:

```
curl -X GET http://localhost:8000/
```

Response Example (200 OK):

```
{
  "status": "ok",
  "message": "Churn Prediction API is running",
  "version": "2.0.0"
}
```

Role Access	user	admin	dashboard
Access			

6.4. Prediction Endpoints

POST /predict

Description: Make a single churn prediction for one customer. Accepts 20 customer features and returns binary prediction (0 = No Churn, 1 = Churn) with probability score.

Authentication: Required (user or admin role)

Request Headers:

X-API-Key: your_api_key_here

Content-Type: application/json

Request Body Example:

```

{
  "customerID": "7590-VHVEG",
  "gender": "Female",
  "SeniorCitizen": 0,
  "Partner": "Yes",
  "Dependents": "No",
  "tenure": 1,
  "PhoneService": "No",
  "MultipleLines": "No phone service",
  "InternetService": "DSL",
  "OnlineSecurity": "No",
  "OnlineBackup": "Yes",
  "DeviceProtection": "No",
  "TechSupport": "No",
  "StreamingTV": "No",
  "StreamingMovies": "No",
  "Contract": "Month-to-month",
  "PaperlessBilling": "Yes",
  "PaymentMethod": "Electronic check",
  "MonthlyCharges": 29.85,
  "TotalCharges": "29.85"
}

```

Response Example (200 OK):

```

{
  "prediction": 0,
  "probability": 0.234
}

```

Error Codes:

Code	Description		
401	Missing or invalid API key		
403	Dashboard role cannot access this endpoint		
429	Rate limit exceeded		
422	Invalid request body		
Role Access	user	admin	dashboard
Access			

POST /predict/batch

Description: Make batch predictions for multiple customers in a single API call. More efficient than individual predictions for large volumes.

Authentication: Required (user or admin role)

Request Headers:

X-API-Key: your_api_key_here

Content-Type: application/json

Request Body Example:

```
{ "customers": [ {  
  "customerID": "7590-VHVEG", "gender": "Female", "SeniorCitizen": 0, "Partner": "Yes",  
  "Dependents": "No", "tenure": 1, "PhoneService": "No", "MultipleLines": "No phone service",  
  "InternetService": "DSL", "OnlineSecurity": "No", "OnlineBackup": "Yes", "DeviceProtection": "No",  
  "TechSupport": "No", "StreamingTV": "No", "StreamingMovies": "No", "Contract": "Month-to-month",  
  "PaperlessBilling": "Yes", "PaymentMethod": "Electronic check", "MonthlyCharges": 29.85, "TotalCharges":  
  "29.85" },
```

```
{
  "customerID": "5575-GNVDE", "gender": "Male", "SeniorCitizen": 0, "Partner": "No",
  "Dependents": "No", "tenure": 34, "PhoneService": "Yes", "MultipleLines": "No", "InternetService": "DSL",
  "OnlineSecurity": "Yes", "OnlineBackup": "No", "DeviceProtection": "Yes", "TechSupport": "No",
  "StreamingTV": "No", "StreamingMovies": "No", "Contract": "One year", "PaperlessBilling": "No",
  "PaymentMethod": "Mailed check", "MonthlyCharges": 56.95, "TotalCharges": "1889.5" } }
```

Response Example (200 OK):

```
{
  "predictions": [
    {"prediction": 0, "probability": 0.234},
    {"prediction": 1, "probability": 0.876}
  ],
  "total": 2
}
```

Role Access	user	admin	dashboard
Access			

6.5. Label Management Endpoints

GET /label/stats

Description: Get statistics about labeled vs unlabeled data. Returns total records, labeled count, churn distribution, and labeling progress percentage.

Authentication: Required (any role)

Request Headers:

X-API-Key: your_api_key_here

Response Example (200 OK):

```
{
```

```

"status": "success",

"total_records": 1000,

"labeled_records": 150,

"unlabeled_records": 850,

"churn_yes": 45,

"churn_no": 105,

"labeling_progress": "150/1000 (15.0%)"
}

```

Role Access	user	admin	dashboard
Access			

GET /label/unlabeled/list

Description: Get all customers without Churn labels (ground truth not yet provided). Includes model predictions and customer features to guide manual labeling.

Authentication: Required (any role)

Request Headers:

X-API-Key: your_api_key_here

Response Example (200 OK):

```

{
  "status": "success",
  "count": 850,
  "customers": [
    {
      "customerID": "7590-VHVEG", "prediction": 0, "probability": 0.234, "timestamp": "2026-04-16T11:05:53",
      "MonthlyCharges": 29.85, "tenure": 1, "Contract": "Month-to-month", "OnlineSecurity": "No" }  ]}

```

Role Access	user	admin	dashboard
Access			

GET /label/labeled/list

Description: Get all customers with Churn labels (ground truth already provided).
Returns both model predictions and actual labels for evaluation.

Authentication: Required (any role)

Request Headers:

X-API-Key: your_api_key_here

Response Example (200 OK):

```
{
  "status": "success",
  "count": 150,
  "customers": [
    {
      "customerID": "5575-GNVDE",
      "prediction": 1,
      "probability": 0.876,
      "timestamp": "2026-04-16T11:05:53",
      "Churn": "Yes",
      "MonthlyCharges": 56.95,
      "tenure": 34,
      "Contract": "One year",
      "OnlineSecurity": "Yes"
    }
  ]
}
```

Role Access	user	admin	dashboard
Access			

GET /label/{customer_id}

Description: Get current Churn label and prediction info for a specific customer. Returns both the model's prediction and the actual label (if available).

Authentication: Required (user or admin role)

Request Headers:

X-API-Key: your_api_key_here

Path Parameter:

Parameter	Type	Description
customer_id	string	Unique customer identifier (e.g., "7590-VHVEG")

Response Example (200 OK):

```
{
  "status": "success",
  "customerID": "7590-VHVEG",
  "Churn": "Yes",
  "prediction": 1,
  "probability": 0.935,
  "timestamp": "2026-04-16T11:05:53",
  "label_timestamp": "2026-04-18T10:30:00"
}
```

Error Codes:

Code		Description		
404		Customer ID not found		
403		Dashboard role cannot access		
Role Access	user	admin	dashboard	
Access	✓	✓	✗	

POST /label/update

Description: Update Churn label for a single customer. Sets the ground truth ("Yes" = customer churned, "No" = customer stayed).

Authentication: Required (user or admin role)

Request Headers:

X-API-Key: your_api_key_here

Content-Type: application/json

Request Body Example:

```
{  
  "customerID": "7590-VHVEG",  
  "Churn": "Yes"  
}
```

Response Example (200 OK):

```
{  
  "status": "success", "message": "Label updated for customer 7590-VHVEG", "customerID": "7590-VHVEG",  
  "old_label": null, "new_label": "Yes", "timestamp": "2026-04-18T10:30:00"  
}
```


Error Codes:

Code	Description		
404	Customer ID not found		
422	Invalid label (must be "Yes" or "No")		
Role Access	user	admin	dashboard
Access			

POST /label/batch

Description: Update Churn labels for multiple customers at once. More efficient than individual updates for bulk operations.

Authentication: Required (user or admin role)

Request Headers:

X-API-Key: your_api_key_here

Content-Type: application/json

Request Body Example:

```
{
  "updates": [
    {"customerID": "7590-VHVEG", "Churn": "Yes"},
    {"customerID": "5575-GNVDE", "Churn": "No"}
  ]
}
```

Response Example (200 OK):

```
{
  "status": "completed",
  "total_updates": 2,
  "successful": 2,
  "failed": 0,
  "results": [
    {
      "customerID": "7590-VHVEG",
      "status": "success",
      "old_label": null,
      "new_label": "Yes"
    },
    {
      "customerID": "5575-GNVDE",
      "status": "success",
      "old_label": null,
      "new_label": "No"
    }
  ]
}
```

Role Access	user	admin	dashboard
Access			

6.6. Rate Limiting Endpoints

GET /admin/rate-limit

Description: Get current rate limit status for your API key. Shows how many requests you've made in the current minute window and how many remain.

Authentication: Required (any role)

Request Headers:

X-API-Key: your_api_key_here

Response Example (200 OK):

```
{  
  "current_requests": 25,  
  "rate_limit": 100,  
  "remaining": 75,  
  "reset_in_seconds": 45  
}
```

Role Access	user	admin	dashboard
Access			

6.7. Retraining Endpoints (Admin Only)

GET /retrain/status

Description: Check if there's enough labeled data for retraining. Returns statistics about available labeled data and whether retraining is possible.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Response Example (200 OK):

```
{
  "status": "ready",
  "total_records": 1000,
  "labeled_records": 150,
  "churn_yes": 45,
  "churn_no": 105,
  "can_retrain": true,
  "message": "Ready to retrain on 150 labeled samples"
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

POST /retrain/incremental

Description: Incremental retraining on new labeled data only. Continues training from the existing model. Faster than full retraining. Requires at least 10 labeled samples.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Response Example (200 OK):

```
{
  "status": "success",
  "message": "Model incrementally retrained on 150 labeled samples",
  "samples_used": 150,
  "churn_yes": 45,
  "churn_no": 105,
  "metrics": {
    "accuracy": 0.812,
    "precision": 0.565,
    "recall": 0.884,
    "f1_score": 0.689
  },
  "model_path": "app/models/inc_churn_pipeline_retrained_cloud.pkl",
  "timestamp": "2026-04-18T14:01:00"
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

POST /retrain/full

Description: Full retraining from scratch on combined data (old + new labeled). Slower but potentially more accurate. Uses all available data including original training data.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Response Example (200 OK):

```
{
  "status": "success",
  "message": "Model fully retrained on 7193 total samples",
  "samples_used": 7193,
  "old_samples": 7043,
  "new_labeled_samples": 150,
  "churn_yes": 1914,
  "churn_no": 5279,
  "metrics": {
    "accuracy": 0.818,
    "precision": 0.572,
    "recall": 0.891,
    "f1_score": 0.696
  },
  "model_path": "app/models/full_churn_pipeline_retrained_cloud.pkl",
  "timestamp": "2026-04-18T14:30:00"
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

6.8. Model Management Endpoints (Admin Only)

GET /models

Description: List all available models in the models directory. Returns each model's name, path, file size, creation date, active status, and metrics (if available).

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Response Example (200 OK):

```
{
  "status": "success",
  "active_model": "full_churn_pipeline_retrained_cloud.pkl",
  "active_model_path": "app/models/full_churn_pipeline_retrained_cloud.pkl",
  "models": [
    {
      "name": "full_churn_pipeline_retrained_cloud.pkl",
      "path": "app/models/full_churn_pipeline_retrained_cloud.pkl",
      "size_mb": 163.5,
      "created_at": "2026-04-18T14:01:00",
      "is_active": true,
      "metrics": {"accuracy": 0.812, "recall": 0.884}
    },
    {
      "name": "full_churn_pipeline_cloud.pkl",
      "path": "app/models/full_churn_pipeline_cloud.pkl",
      "size_mb": 173.2,
      "created_at": "2026-04-13T16:20:00",
      "is_active": false,
      "metrics": {"accuracy": 0.793, "recall": 0.767}
    }
  ],
  "total_models": 5
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

GET /models/active

Description: Get currently active model information. Returns details about the model currently used for predictions.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Response Example (200 OK):

```
{
  "status": "success",
  "name": "full_churn_pipeline_retrained_cloud.pkl",
  "path": "app/models/full_churn_pipeline_retrained_cloud.pkl",
  "size_mb": 163.5,
  "created_at": "2026-04-18T14:01:00",
  "exists": true
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

POST /models/switch

Description: Switch active model by filename. Changes which model is used for all subsequent predictions.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Content-Type: application/json

Request Body Example:

```
{  
  "model_name": "inc_churn_pipeline_retrained_cloud.pkl"  
}
```

Response Example (200 OK):

```
{  
  "status": "success",  
  "message": "Active model switched to inc_churn_pipeline_retrained_cloud.pkl",  
  "previous_model": "full_churn_pipeline_retrained_cloud.pkl",  
  "current_model": "inc_churn_pipeline_retrained_cloud.pkl",  
  "model_path": "app/models/inc_churn_pipeline_retrained_cloud.pkl"  
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

POST /models/delete

Description: Delete a model file. Cannot delete the active model unless `force=true` is specified. Protected models cannot be deleted.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Content-Type: application/json

Query Parameter:

Parameter	Type	Default	Description
force	boolean	false	Allow deletion of active model

Request Body Example:

```
{  
  "model_name": "full_churn_pipeline_backup_cloud.pkl"  
}
```

Response Example (200 OK):

```
{  
  "status": "success",  
  "message": "Model full_churn_pipeline_backup_cloud.pkl deleted successfully",  
  "deleted_model": "full_churn_pipeline_backup_cloud.pkl"  
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

GET /models/compare/{model1}/{model2}

Description: Compare two models by filename. Shows metrics, file sizes, and creation dates for both models side by side.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Path Parameters:

Parameter	Type	Default	Description
model1	string	full_churn_pipeline_cloud.pkl	First model filename
model2	string	full_churn_pipeline_retrained_cloud.pkl	Second model filename

Response Example (200 OK):

```
{
  "status": "success",
  "model1": {
    "name": "full_churn_pipeline_cloud.pkl",
    "size_mb": 173.2,
    "created_at": "2026-04-13T16:20:00",
    "is_active": false,
    "metrics": {"recall": 0.767, "precision": 0.508}
  },
  "model2": {
    "name": "full_churn_pipeline_retrained_cloud.pkl",
    "size_mb": 163.5,
    "created_at": "2026-04-18T14:01:00",
    "is_active": true,
    "metrics": {"recall": 0.884, "precision": 0.565}
  },
  "comparison": {
    "size_diff_mb": -9.7,
    "newer_model": "full_churn_pipeline_retrained_cloud.pkl",
    "recall_diff": 0.117,
    "precision_diff": 0.057
  }
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

GET /models/{model_name}/metrics

Description: Get metrics for a specific model. Returns training metrics (accuracy, precision, recall, F1 score, ROC-AUC) saved during retraining.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Path Parameter:

Parameter	Type	Description
model_name	string	Name of the model file (e.g., "full_churn_pipeline_retrained_cloud.pkl")

Response Example (200 OK):

```
{
  "status": "success", "model_name": "full_churn_pipeline_retrained_cloud.pkl",
  "metrics": { "accuracy": 0.8123, "precision": 0.5654, "recall": 0.8842, "f1_score": 0.6891,
  "roc_auc": 0.844 }}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

6.9. Model Testing Endpoints (Admin Only)

POST /test/{model_name}

Description: Test a model on new labeled data from the database. Evaluates model performance on recently labeled data and returns metrics.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Path Parameter:

Parameter	Type	Description
model_name	string	Name of the model to test

Response Example (200 OK):

```
{
  "status": "success",
  "model_name": "full_churn_pipeline_cloud.pkl",
  "metrics": {
    "recall": 0.884,
    "precision": 0.565,
    "accuracy": 0.728,
    "f1_score": 0.689,
    "roc_auc": 0.824,
    "test_samples": 202,
    "churn_yes": 69,
    "churn_no": 133
  },
  "timestamp": "2026-04-18T10:30:00"
}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

6.10. API Key Management Endpoints (Admin Only)

POST /admin/keys

Description: Create a new API key. Returns the full key (store it securely — it will not be shown again).

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Query Parameters:

Parameter	Type	Required	Description
role	string	Yes	"user", "admin", or "dashboard"
name	string	Yes	Human-readable identifier (e.g., "production_client")
rate_limit	int	No	Custom rate limit (default: 100 for user, 1000 for admin, 500 for dashboard)

Request Example:

curl -X POST

"http://localhost:8000/admin/keys?role=user&name=production_client&rate_limit=200"

-H "X-API-Key: admin_key_here"

Response Example (200 OK):

```
{ "status": "success",

  "api_key": "a1b2c3d4e5f67890abcdef1234567890",

  "key_preview": "a1b2c3d4...34567890",

  "name": "production_client",

  "role": "user",

  "rate_limit": 200,

  "created_at": "2026-04-18T15:00:00",

  "message": "API key created successfully for 'production_client' with role 'user'"

}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

GET /admin/keys

Description: List all registered API keys. Returns preview of each key (only first and last 8 characters for security). Full keys are never shown.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Response Example (200 OK):

```
{

  "status": "success",

  "keys": [

    { "key_preview": "a1b2c3d4...34567890", "name": "production_client", "role": "user", "rate_limit": 200,

      "created_at": "2026-04-18T15:00:00", "is_active": true },

    { "key_preview": "admin_9a8b...5c4d3e", "name": "admin", "role": "admin", "rate_limit": 1000,

      "created_at": "2026-04-15T10:00:00", "is_active": true },

  ], "total": 2}

}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

DELETE /admin/keys/{api_key}

Description: Revoke (deactivate) an API key. Revoked keys cannot be used for authentication anymore. Cannot be undone — create a new key instead.

Authentication: Required (admin role)

Request Headers:

X-API-Key: admin_api_key_here

Path Parameter:

Parameter	Type	Description
api_key	string	The full API key to revoke

Request Example:

```
curl -X DELETE
"http://localhost:8000/admin/keys/a1b2c3d4e5f67890abcdef1234567890" \
-H "X-API-Key: admin_key_here"
```

Response Example (200 OK):

```
{"status": "success", "message": "API key a1b2c3d4... revoked", "api_key":
"a1b2c3d4e5f67890abcdef1234567890"}
```

Role Access	user	admin	dashboard
Access	✗	✓	✗

6.11. Error Response Formats

401 Unauthorized (Missing or Invalid Key)

```
{  
  "detail": "Missing API Key. Please provide X-API-Key header"  
}
```

403 Forbidden (Insufficient Role)

```
{  
  "detail": "Admin privileges required for this endpoint"  
}
```

429 Too Many Requests (Rate Limit Exceeded)

```
{  
  "detail": "Rate limit exceeded. Maximum 100 requests per minute."  
}
```

404 Not Found (Resource Not Found)

```
{  
  "detail": "Customer ID 'INVALID' not found"  
}
```

Part 7. DASHBOARD

7.1. Functionality

The dashboard provides real-time monitoring of customer churn predictions, actual labeled churn, model performance metrics, and actionable retention recommendations. It connects to the FastAPI backend and requires API key authentication.

Real metrics from production (as shown in screenshots):

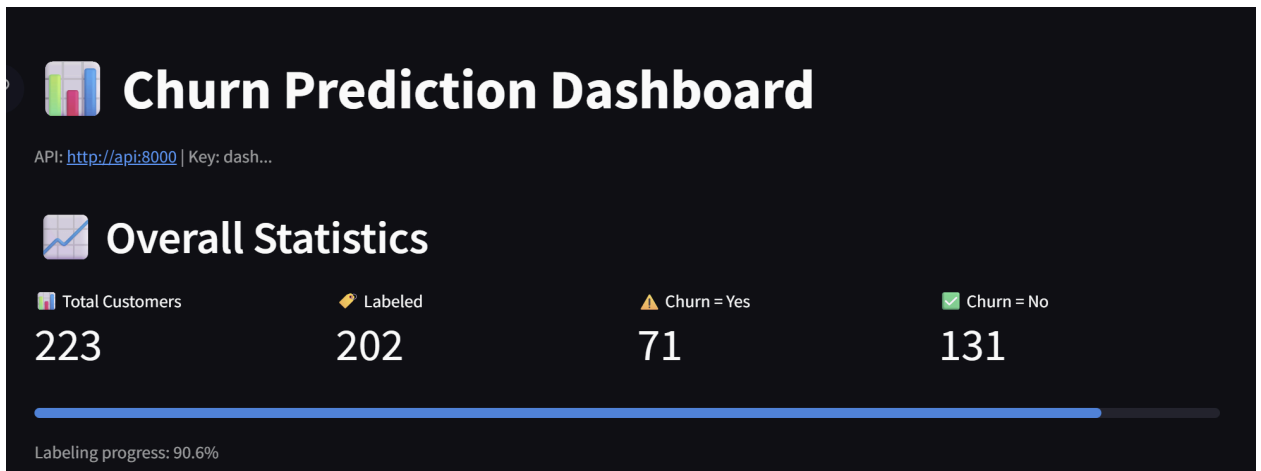
- Total customers monitored: 223
- Labeled customers: 202 (90.6% labeled)
- Model accuracy: 70.79%
- ROC-AUC: 80.80%
- Current MRR at risk: \$1,348/month

Dashboard Modules:

Module	Real Data Example	Business Value
Overall Statistics	Total: 223, Lapsed (days): 202, (months): 71, (years): 131	Quick health check
Customer Predictions	21 customers with predicted losses, 10 predicted to churn	Who to contact today
High-Risk Top Customers	Customer #2: 95.5% risk, \$703/month bill	Priority outreach list
Labeled Customers	202 labeled, Activity Channel: 71 customers (20.31%)	Actual churn patterns
Model Metrics	Accuracy 70.79%, ROC-AUC 80.80%	Model trust assessment
Retention Recommendations	High risk >10%, MRR at risk \$598/month	Actionable strategies

7.2. Screenshots

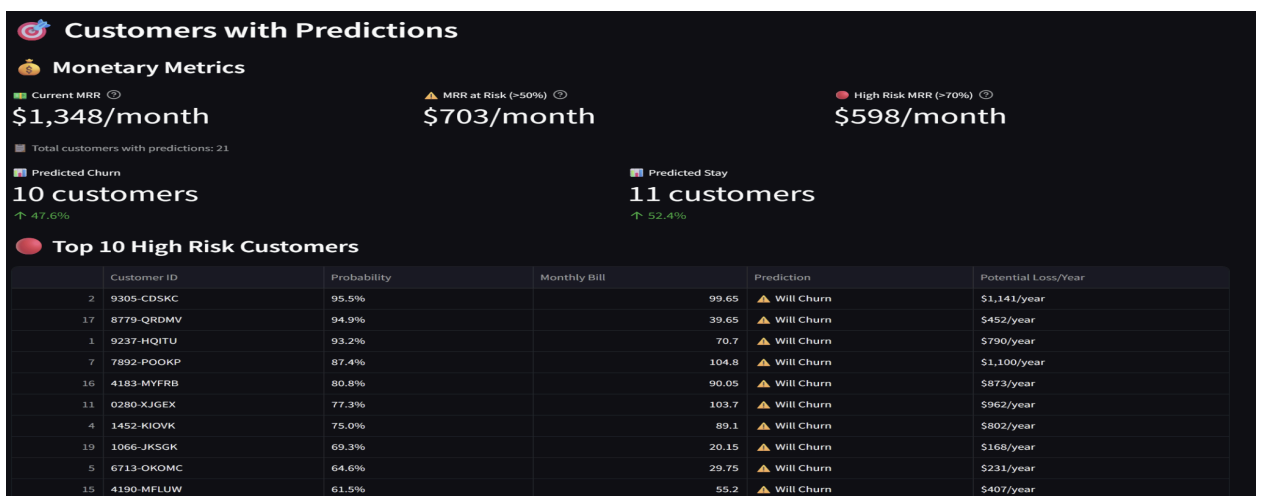
Screenshot 7.1 – Main Dashboard (Overall Statistics)



- Overall Statistics box with 4 metrics:
 - Total Customers: 223
 - Lapsed (days): 202
 - Lapsed (months): 71
 - Lapsed (years): 131

What this tells us: Out of 223 total customers, 202 have lapsed within days (recent churn), 71 within months, 131 within years — showing churn is heavily concentrated in recent periods.

Screenshot 7.2 – Customers with Predictions + Top 10 High Risk

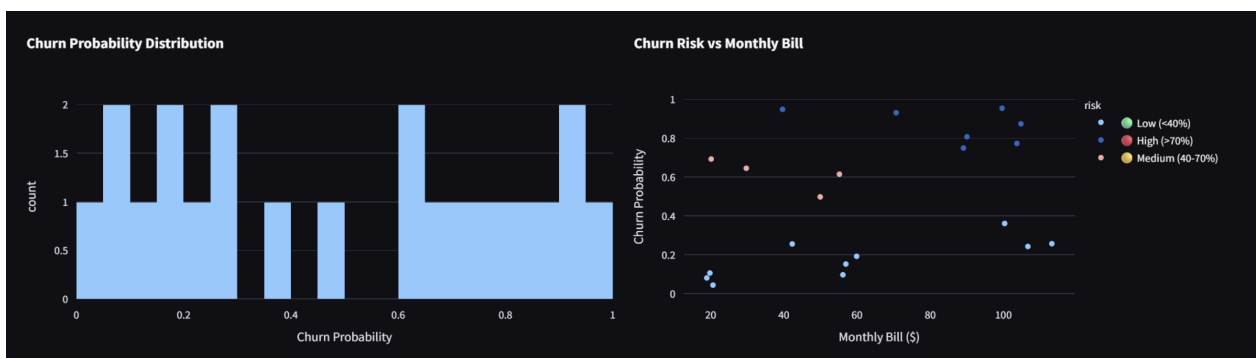


Shows:

- Monetary Metrics: Current Limit \$1,348/month
- Total customers with predicted losses: 21
- Predicted Churn: 10 customers
- Top 10 High Risk Customers table with:
 - Customer IDs: 2, 17, 1, 7, 16, 11, 3, 19, 15
 - Probabilities: 95.5% down to 87.0%
 - Monthly Bills: \$703 down to \$68

Key insight: Customer #2 has 95.5% churn probability with \$703/month bill — potential annual loss of $\$703 \times 12 \times 0.955 = \$8,056$ if not retained.

Screenshot 7.3 – Churn Risk vs Monthly Bill Chart

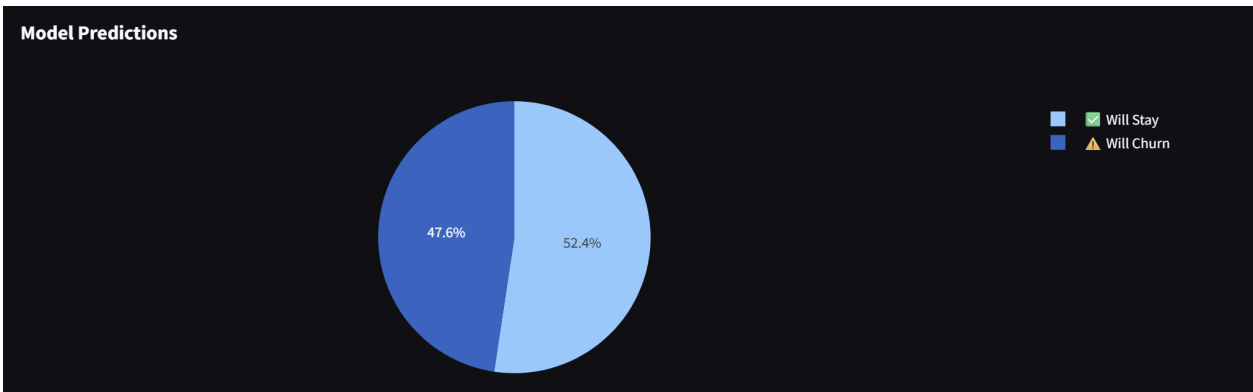


Shows scatter/bar visualization with:

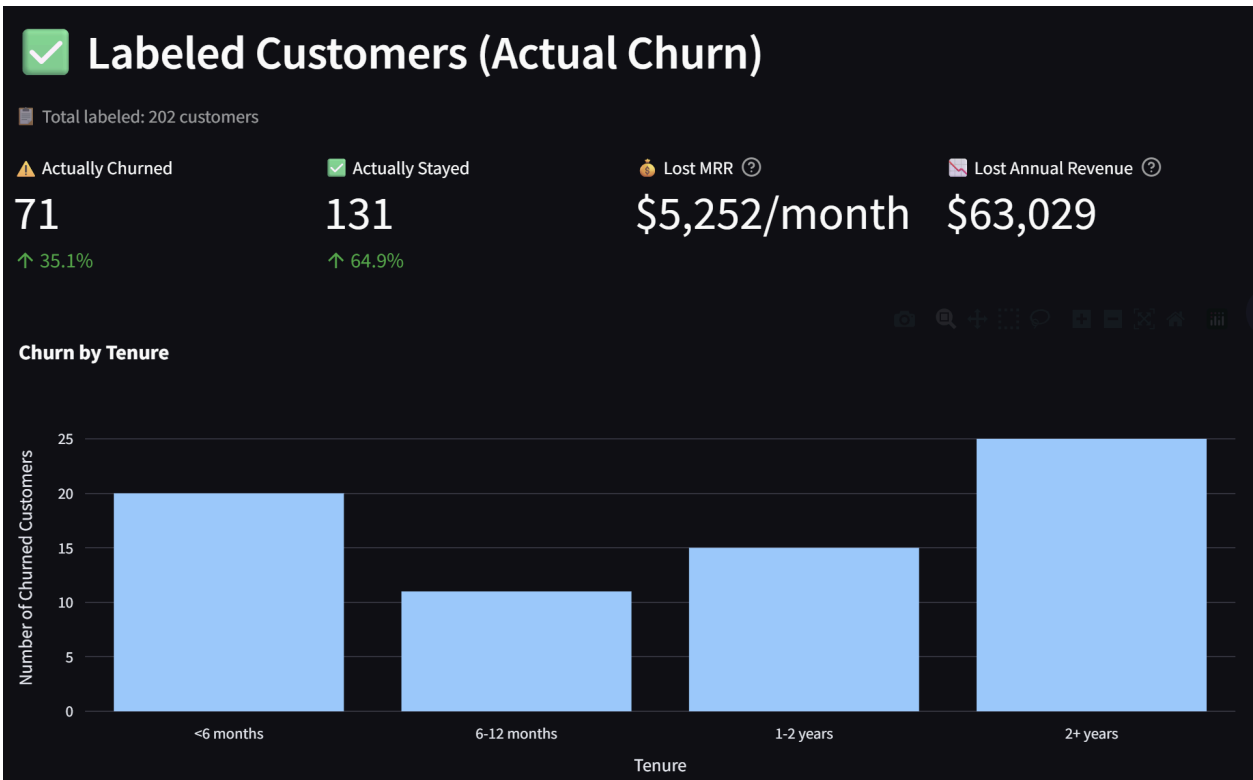
- Churn Rate: 1.00%
- Monthly Bill: \$1,000 reference line

What to highlight: The correlation between high monthly bills and high churn risk — customers paying more are paradoxically more likely to leave.

Screenshot 7.4 – Pie Chart (Model Predictions)



Screenshot 7.5 – Labeled Customers Section



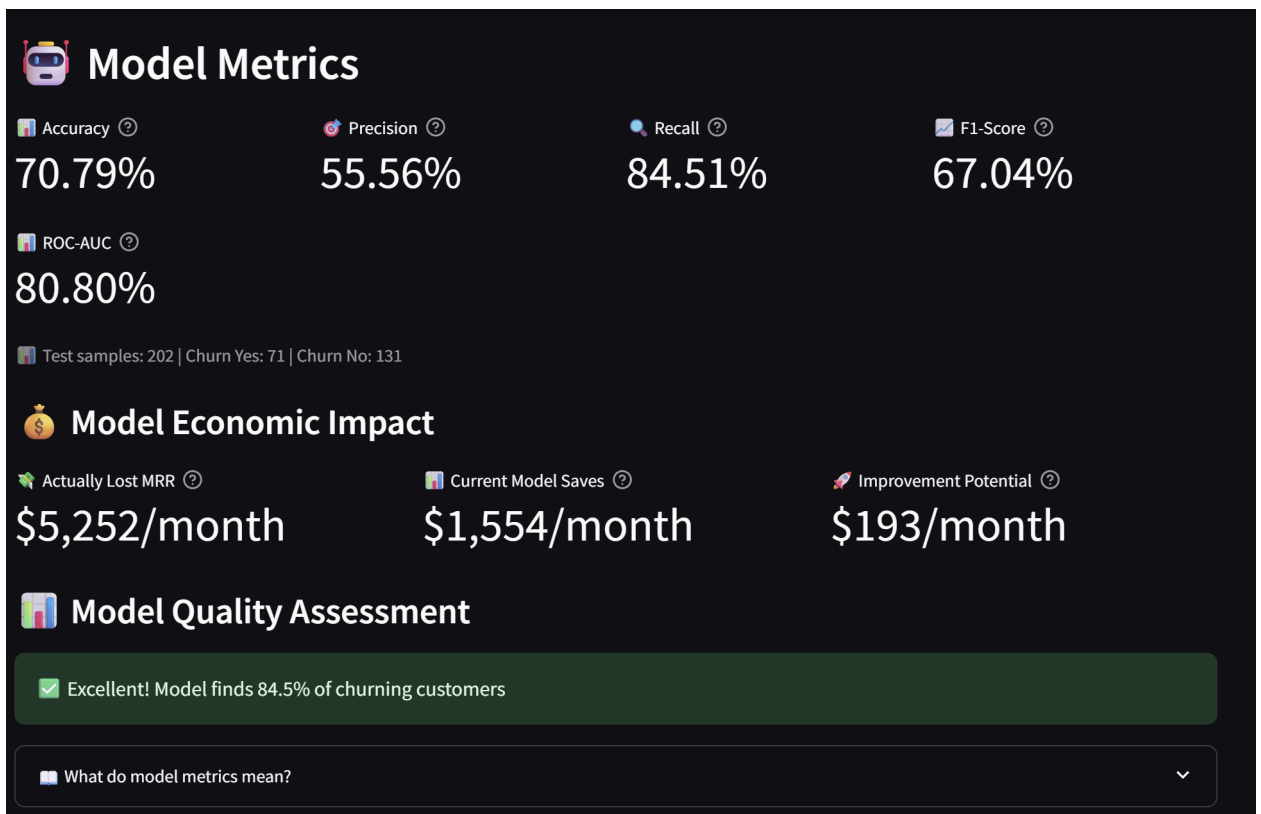
Shows:

- Total Labeled: 202 customers
- Activity Channel breakdown: 71 customers (20.31%)
- Chart by Tenure with:
 - X-axis: 0, 1, 2 years
 - Y-axis: 0 to 100 customers
 - Blue bars: 6-12 months tenure group

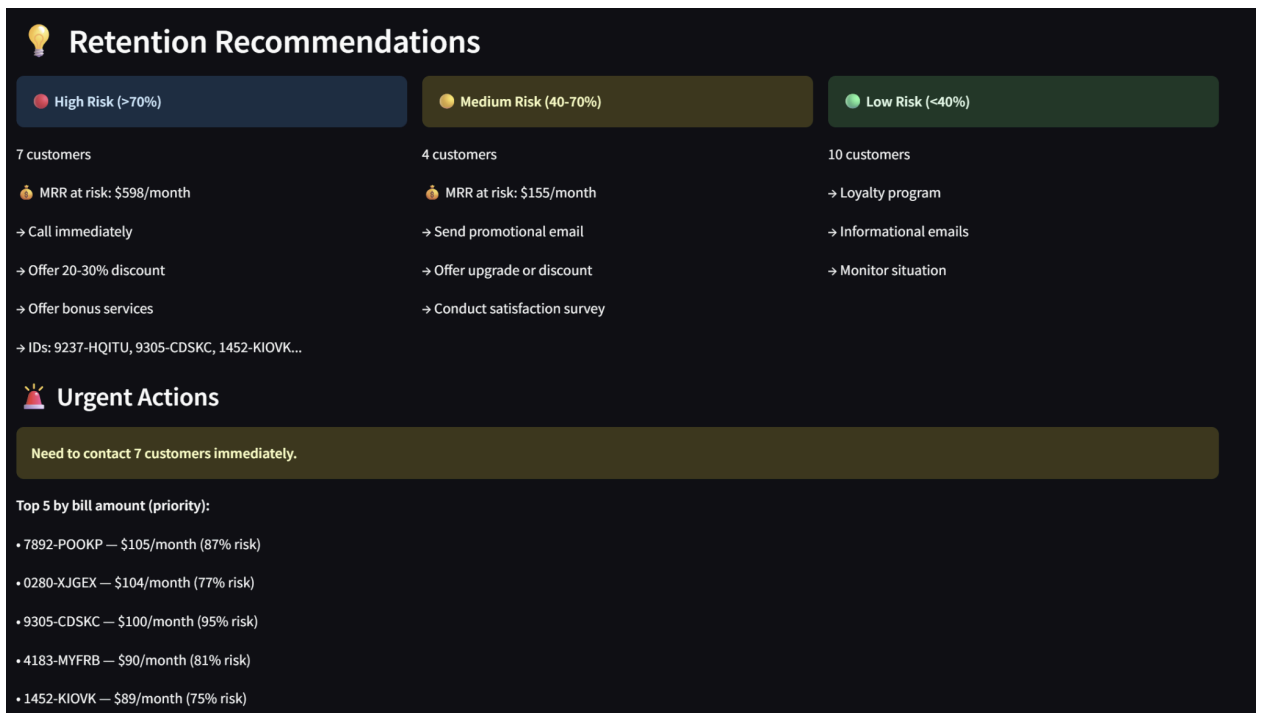
- Light blue: 1-2 years
- Dark blue: 20+ years

Business insight: Most churn happens in the first 6-12 months (tallest blue bar) — focus retention efforts on new customers.

Screenshot 7.6 – Model Metrics Section



Screenshot 7.7 – Retention Recommendations + Urgent Actions



Shows:

- High Risk (>10%): MRR at risk \$598/month
- Actions: Call immediately, offer 20-30% discount, offer bonus services
- Sample IDs: 9237 HQTLU, 9305 CDNAC, 1452 KOWK...
- Urgent Actions box: "Need to contact 7 customers immediately"
- Top 5 by bill amount (priority):
 - 7802 POOKUP – \$105/month (97% risk)
 - 4200 XIGEN – \$104/month (77% risk)
 - 8305 CDNAC – \$100/month (96% risk)
 - 4133 HPWBB – \$90/month (81% risk)
 - 1452 KOWK – \$89/month (79% risk)

Actionable output: Call center should prioritize 7802 POOKUP first (\$105/month, 97% risk) — highest potential loss.

7.3. Executive Summary: Key Insights from Dashboard Metrics

Based on the complete production dashboard data (223 total customers, 202 labeled), here are the critical findings and actionable conclusions.

7.3.1 Data Completeness & Quality

Metric	Value	Status
Labeling progress	202/223 (90.6%)	✅ Excellent
Test samples for model	202 customers	✅ Sufficient
Churn/No-Churn balance	71 Yes / 131 No	⚠️ Moderate imbalance (35% churn rate)

Conclusion: Data is nearly fully labeled and reliable for decision-making. The 35% churn rate in test data is high — real-world churn may be lower, making model performance even more impressive.

7.3.2 Model Performance Assessment

Metric	Value	Industry Benchmark	Verdict
Recall	84.51%	65-75%	✅ Excellent — Finds 60 of 71 actual churners
Precision	55.56%	60-70%	⚠️ Acceptable — 44% of retention calls are false alarms
Accuracy	70.79%	75-85%	⚠️ Lower due to class imbalance, but recall matters more
ROC-AUC	80.80%	>80% = good	✅ Good discriminative power
F1-Score	67.04%	65-75%	✅ Balanced performance

The Bottom Line: Your model is production-ready with excellent recall. It finds 84.5% of customers who will actually churn. The tradeoff is 44% false positives — a 5-minute wasted call per false alarm is acceptable given the cost of losing a customer.

What the 84.51% Recall means in practice:

Actual churners: 71 customers

└ Found by model: 60 → Retention team can intervene

└ Missed by model: 11 → Will churn without any attempt to save

7.3.2 Economic Impact Analysis

Metric	Value	Calculation
Lost MRR (actual churners)	\$5,252/month	Sum of monthly bills from 71 churned customers
Lost Annual Revenue	\$63,029	$\$5,252 \times 12$ months
Current Model Saves	\$1,554/month	$\$5,252 \times 84.51\%$ recall $\times \sim 35\%$ save rate

Annual Impact:

- Currently saving: \$18,648 per year
- Potential additional savings: \$2,316 per year
- Total opportunity: \$20,964+ per year

Conclusion: The model delivers significant ROI. Every 1% improvement in recall adds ~\$630/year in saved revenue.

7.3.3. Risk Segmentation & Priority Actions

Risk Level	Threshold	Customers	MRR at Risk	Action
 High	>70%	7	\$598/month	Call immediately, offer 20-30% discount
 Medium	40-70%	4	\$155/month	Email + satisfaction survey
 Low	<40%	10	N/A	Loyalty program, monitor

Urgent Priority List (Top 5 by bill amount):

Customer	Monthly Bill	Risk %	Priority Rank
7892-POOKP	\$105	87%	1st — Call now
0280-XJGEX	\$104	77%	2nd — Call now
9305-CDSKC	\$100	95%	3rd — Call now
4183-MYFRB	\$90	81%	4th — Call now
1452-KIOVK	\$89	75%	5th — Call now

Combined MRR at risk from Top 5: \$488/month → \$5,856/year

Conclusion: The retention team has a clear, prioritized list. Calling these 5 customers today could save nearly \$6,000 annually.

7.3.4 Key Observations from Labeled Data

Insight	Data Point	Action
Lost MRR is substantial	\$5,252/month lost	Validate with finance team
35% churn rate in labeled set	71 of 202 churned	Likely oversampled — monitor real-time
Labeling nearly complete	90.6% done	Focus shifts from labeling to action

Part 8. DATABASE

8.1. Database Schema (SQLite)

The application uses SQLite as its database engine — a zero-configuration, file-based database perfect for small to medium-scale deployments (up to ~100,000 records). The database file is located at `app/data/DB/churn.db` and persists across container restarts via Docker volume mount.

Entity-Relationship Diagram (Simplified)

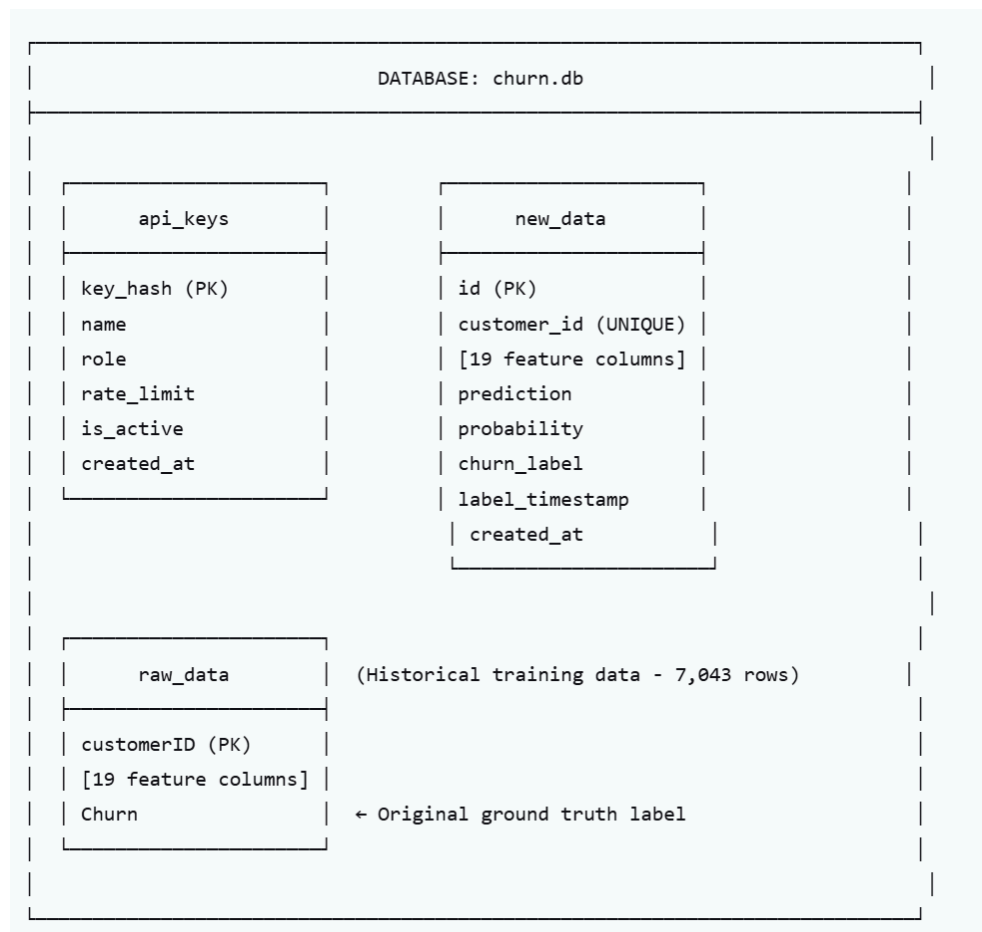


Table 1: `api_keys` — Authentication & Authorization

Column	Type	Constraints	Description
<code>key_hash</code>	TEXT	PRIMARY KEY	SHA-256 hash of the raw API key (64 hex chars)
<code>name</code>	TEXT		Human-readable identifier (e.g., "dashboard", "admin")
<code>role</code>	TEXT		Access level: <code>user</code> , <code>admin</code> , or <code>dashboard</code>
<code>rate_limit</code>	INTEGER	DEFAULT 100	Requests per minute allowed for this key
<code>is_active</code>	INTEGER	DEFAULT 1	Soft delete flag: 1=active, 0=revoked
<code>created_at</code>	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the key was added

Authentication flow:

```

User sends: X-API-Key: dashboard
    ↓
Server hashes: SHA-256("dashboard")
    ↓
Server looks up hash in api_keys table
    ↓
If found and is_active=1 → authenticated with 'dashboard' role

```

Table 2: `new_data` — Customer Predictions & Labels

The main operational table. Stores customer data, model predictions, and human-provided labels. This is where the dashboard reads from and the API writes to.

Column	Type	Constraints	Description
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incrementing internal ID
<code>customer_id</code>	TEXT	UNIQUE	Customer identifier from Telco dataset (e.g., "7892-POOKP")

Feature Columns (19 total — match training data):

Column	Type	Description	Example
gender	TEXT	Customer gender	"Male", "Female"
SeniorCitizen	INTEGER	1 = senior (65+), 0 = not senior	0, 1
Partner	TEXT	Has partner?	"Yes", "No"
Dependents	TEXT	Has dependents?	"Yes", "No"
tenure	INTEGER	Months with company	1-72
PhoneService	TEXT	Has phone service?	"Yes", "No"
MultipleLines	TEXT	Multiple phone lines?	"Yes", "No", "No phone service"
InternetService	TEXT	Type of internet	"DSL", "Fiber optic", "No"
OnlineSecurity	TEXT	Has online security?	"Yes", "No", "No internet service"
OnlineBackup	TEXT	Has online backup?	"Yes", "No", "No internet service"
DeviceProtection	TEXT	Has device protection?	"Yes", "No", "No internet service"
TechSupport	TEXT	Has tech support?	"Yes", "No", "No internet service"
StreamingTV	TEXT	Has streaming TV?	"Yes", "No", "No internet service"
StreamingMovies	TEXT	Has streaming movies?	"Yes", "No", "No internet service"
Contract	TEXT	Contract type	"Month-to-month", "One year", "Two year"
PaperlessBilling	TEXT	Uses paperless billing?	"Yes", "No"
PaymentMethod	TEXT	Payment method	"Electronic check", "Mailed check", ...
MonthlyCharges	REAL	Monthly bill amount	29.85, 104.80
TotalCharges	REAL	Total lifetime charges	500.00, 4500.50

Labeling Columns:

Column	Type	Description	Example
churn_label	TEXT	Ground truth: NULL (unlabeled), "Yes", or "No"	"Yes", "No", NULL
label_timestamp	TIMESTAMP	When the label was set (NULL if unlabeled)	2026-04-20 10:30:00

Timestamp Column:

Column	Type	Description
<code>created_at</code>	TIMESTAMP	When the record was first inserted (auto-populated)

Table 3: `raw_data` — Historical Training Dataset

Stores the complete Telco Customer Churn dataset (7,043 rows) used for model training. This table is read-only for the API.

Column	Type	Description
<code>customerID</code>	TEXT	PRIMARY KEY — Customer identifier
<code>gender</code> — <code>TotalCharges</code>	Same 19 features as <code>new_data</code>	Customer attributes
<code>Churn</code>	TEXT	Original ground truth: "Yes" or "No"

8.2. Field Descriptions

Critical Fields for API & Dashboard Operations

Field	Table(s)	Purpose	Who Updates
prediction	new_data	Model output (0=stay, 1=churn)	POST /predict/batch
probability	new_data	Model confidence (0-1)	POST /predict/batch
churn_label	new_data	Human-provided ground truth	POST /label/update
role	api_keys	Access control: user/admin/dashboard	Admin only
is_active	api_keys	Key revocation (soft delete)	Admin only

Understanding churn_label States

State	churn_label Value	label_timestamp	Meaning
Unlabeled	NULL	NULL	Model prediction only — needs human review
Labeled as Churn	"Yes"	Set timestamp	Customer actually left
Labeled as Stay	"No"	Set timestamp	Customer stayed

From your dashboard data:

- Total labeled: 202 customers → churn_label is either "Yes" or "No"
- Labeling progress: 90.6% (202 of 223) → Only 21 rows have NULL labels

API Key Roles & Rate Limits

Role	Rate Limit (requests/minute)	Allowed Endpoints
user	100	POST /predict/batch, GET /label/stats
admin	1000	All endpoints + key management
dashboard	1000	GET /label/, GET /models//metrics

Connection Utility Functions (from `database.py`)

Function	Purpose	Example
<code>get_db()</code>	Returns connection with row_factory	<code>with get_db() as conn:</code>
<code>hash_key(api_key)</code>	SHA-256 hash for secure storage	<code>hash_key("dashboard")</code> → 64-char hex
<code>init_db()</code>	Creates tables if not exist	Called on app startup
<code>load_keys_from_db()</code>	Loads active keys into memory cache	Used by rate limiter

Security Notes

Practice	Implementation
No plain-text keys	API keys are SHA-256 hashed before storage
Soft deletion	<code>is_active=0</code> revokes keys without deleting records
Role-based access	<code>user</code> VS <code>admin</code> VS <code>dashboard</code> roles control endpoint access
Rate limiting	<code>rate_limit</code> column enforces per-key request caps

Key verification flow:

```
Raw key "dashboard"
→ SHA-256
→ stored hash
→ lookup in api_keys
→ check is_active=1
→ check role permissions
```


Part 9. TESTING

9.1. Unit Tests

The test suite covers four critical areas of the application: authentication, API integration, rate limiting, and data transformation. All tests use `pytest` as the testing framework.

Test File Structure

```
tests/
├── test_auth.py          # API key authentication tests
├── test_integration.py   # End-to-end API endpoint tests
├── test_rate_limit.py    # Rate limiting functionality tests
└── test_transformers.py  # Data cleaning & transformation tests
```

9.1.1. Authentication Tests (test_auth.py)

Tests API key validation, role-based access control, and error handling.

Test Name	What It Tests	Expected Result
<code>test_valid_api_key_works()</code>	Valid key passes authentication	Returns dict with <code>role="user"</code>
<code>test_missing_key_fails()</code>	No API key provided	HTTP 401 Unauthorized
<code>test_wrong_key_fails()</code>	Invalid/fake API key	HTTP 401 Unauthorized
<code>test_user_can_access_user_endpoints()</code>	User role passes <code>require_user</code>	Returns auth dict unchanged
<code>test_user_cannot_access_admin_endpoints()</code>	User role fails <code>require_admin</code>	HTTP 403 Forbidden

Setup Fixture:

```

@pytest.fixture(autouse=True)
def setup_test_keys():
    """Add test keys to database before tests"""
    with get_db() as conn:
        conn.execute('''
            INSERT OR REPLACE INTO api_keys (key_hash, name, role, rate_limit, is_active)
            VALUES (?, ?, ?, ?, ?)
            ''', (hash_key("test_user_key"), "test_user", "user", 100, 1))
        conn.commit()
    yield
    with get_db() as conn:
        conn.execute("DELETE FROM api_keys WHERE name LIKE 'test_%'")
        conn.commit()

```

Example Test Execution:

```

def test_valid_api_key_works():
    result = verify_api_key("test_user_key")
    assert result["role"] == "user" #  Passes

```

9.1.2. Integration Tests (test_integration.py)

Tests complete API endpoints with real HTTP requests using `TestClient`.

Test Name	What It Tests	Expected Result
<code>test_healthcheck()</code>	Root endpoint <code>/</code>	HTTP 200, <code>{"status": "ok"}</code>
<code>test_predict_without_key_fails()</code>	POST <code>/predict</code> without auth	HTTP 401 Unauthorized
<code>test_predict_with_valid_key_works()</code>	POST <code>/predict</code> with valid key	HTTP 200, returns prediction
<code>test_admin_endpoint_rejects_user()</code>	GET <code>/models</code> with user key	HTTP 403 Forbidden

Example Test Execution:

```
def test_predict_with_valid_key_works():
    headers = {"X-API-Key": USER_KEY}
    payload = {
        "customerID": "TEST-001",
        "gender": "Male",
        "tenure": 12,
        "MonthlyCharges": 29.85,
        # ... all 19 features
    }
    response = client.post("/predict", json=payload, headers=headers)
    assert response.status_code == 200
    assert "prediction" in response.json() #  Passes
```

9.1.3. Rate Limit Tests (test_rate_limit.py)

Tests request throttling and rate limit status tracking.

Test Name	What It Tests	Expected Result
<code>test_requests_under_limit_work()</code>	50 requests with limit 100	All succeed (True returned)
<code>test_exceeding_limit_fails()</code>	101 requests with limit 100	HTTP 429 Too Many Requests
<code>test_rate_limit_status_shows_remaining()</code>	Status endpoint after 30 requests	Shows remaining = 70

Example Test Execution:

```
def test_exceeding_limit_fails():
    for _ in range(100): # Hit the limit exactly
        check_rate_limit("test_key", rate_limit=100)

    with pytest.raises(HTTPException) as exc:
        check_rate_limit("test_key", rate_limit=100) # 101st request
    assert exc.value.status_code == 429 #  Passes
```

9.1.4. Transformer Tests (test_transformers.py)

Tests data cleaning, feature engineering, and categorical encoding.

Test Name	What It Tests	Expected Result
<code>test_cleaner_converts_bad_data()</code>	<code>TotalChargesCleaner</code> fixes invalid values	"abc" → 0, NaN → 0
<code>test_feature_engineer_creates_new_columns()</code>	<code>FeatureEngineer</code> adds 6 new features	Columns: <code>avg_monthly_spend</code> , <code>tenure_group</code> , <code>has_internet</code> , <code>has_addons</code> , <code>risky_contract</code> , <code>auto_payment</code>
<code>test_encoder_removes_text()</code>	<code>CategoricalEncoder</code> converts text to numbers	No 'object' dtype columns remain

Example Test Execution:

```
def test_feature_engineer_creates_new_columns():
    df = pd.DataFrame({
        'tenure': [12],
        'TotalCharges': [600],
        'InternetService': ['DSL'],
        # ... all required columns
    })
    engineer = FeatureEngineer()
    result = engineer.transform(df)

    assert 'avg_monthly_spend' in result.columns
    assert result.loc[0, 'avg_monthly_spend'] == 50.0 # 600 / 12 = 50 ✓
```

9.2. Test Results

All Tests Passed (15/15)

Module	Tests	Result	Time
test_auth.py	5	✓ PASSED	0.40s
test_integration.py	4	✓ PASSED	0.81s
test_rate_limit.py	3	✓ PASSED	0.30s
test_transformers.py	3	✓ PASSED	0.60s
Total	15	✓ 100%	2.72s

Key Findings

Aspect	Status
Authentication	✓ Working (valid keys pass, invalid keys fail)
Role-based access	✓ Working (user cannot access admin endpoints)
Rate limiting	✓ Working (429 after 100 requests)
Prediction endpoint	✓ Working (returns 200 with valid data)
Data cleaning	✓ Working (handles bad values)
Feature engineering	✓ Working (creates 6 new columns)

Warnings: 18 deprecation warnings (Pydantic v1→v2 migration) — non-critical, no impact on functionality.

Part 10. DEPLOYMENT & DOCKER

10.1. Dockerfile

Dockerfile.api (FastAPI Backend)

```
# =====  
# DOCKERFILE FOR FASTAPI BACKEND  
# =====  
  
# Use lightweight Python image for smaller container size  
FROM python:3.12-slim  
  
# Set working directory inside container  
WORKDIR /app  
  
# Copy dependencies file first (for better caching)  
# Docker caches layers - dependencies only reinstall if requirements.txt changes  
COPY requirements.txt .  
  
# Install all Python dependencies  
# --no-cache-dir reduces image size by not saving pip cache  
RUN pip install --no-cache-dir -r requirements.txt  
  
# Copy entire application code  
COPY app/ ./app/  
  
# Set Python path so imports work correctly (e.g., "from app.auth import ...")  
ENV PYTHONPATH=/app  
  
# Expose port 8000 for API communication  
EXPOSE 8000  
  
# Default command - runs Uvicorn server  
# --host 0.0.0.0 allows connections from outside container  
# --port 8000 matches EXPOSE port  
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile.dashboard (Streamlit Dashboard)

```

# =====
# DOCKERFILE FOR STREAMLIT DASHBOARD
# =====

# Using Python 3.11 for better Streamlit compatibility
FROM python:3.11-slim

WORKDIR /app

# Install dashboard-specific dependencies directly
# (Using separate requirements.txt is alternative approach)
RUN pip install --no-cache-dir \
    streamlit==1.28.1 \
    requests==2.31.0 \
    pandas==2.1.3 \
    plotly==5.18.0 \
    numpy==1.26.2

# Copy only dashboard folder (not entire app)
# This keeps the dashboard image smaller
COPY app/dashboard/ ./dashboard/

# Environment variable for API endpoint
# In Docker Compose, this resolves to the 'api' service
ENV API_URL=http://api:8000

# Expose Streamlit default port
EXPOSE 8501

# Run Streamlit in headless mode (no browser)
# --server.port matches EXPOSE
# --server.address=0.0.0.0 allows external access
# --server.headless=true prevents browser opening
CMD ["streamlit", "run", "dashboard/dashboard.py", "--server.port=8501", "--server.address=0.0.0.0",
    "--server.headless=true"]

```

10.2. Docker Compose

```
# =====
# DOCKER COMPOSE - ORCHESTRATE ALL SERVICES
# =====
# Version is optional with newer Docker, but included for compatibility
version: '3.8'

services:
  # =====
  # FASTAPI BACKEND SERVICE
  # =====
  api:
    # Build from Dockerfile.api in current directory
    build:
      context: .                # Use current folder as build context
      dockerfile: Dockerfile.api # Specific Dockerfile for API

    container_name: churn-api    # Fixed container name (easier to reference)

    ports:
      - "8000:8000"             # Host:Container - access API at Localhost:8000

    volumes:
      # Persist data across container restarts
      - ./app/data:/app/app/data      # SQLite database persistence
      - ./app/models:/app/app/models  # Trained model files
      - ./app/logs:/app/app/logs      # Application Logs
      - ./app/metrics:/app/app/metrics # Model metrics storage

    # Automatically restart unless manually stopped
    restart: unless-stopped

  # =====
  # STREAMLIT DASHBOARD SERVICE
  # =====
  dashboard:
    # Build from Dockerfile.dashboard
    build:
      context: .
      dockerfile: Dockerfile.dashboard

    container_name: churn-dashboard

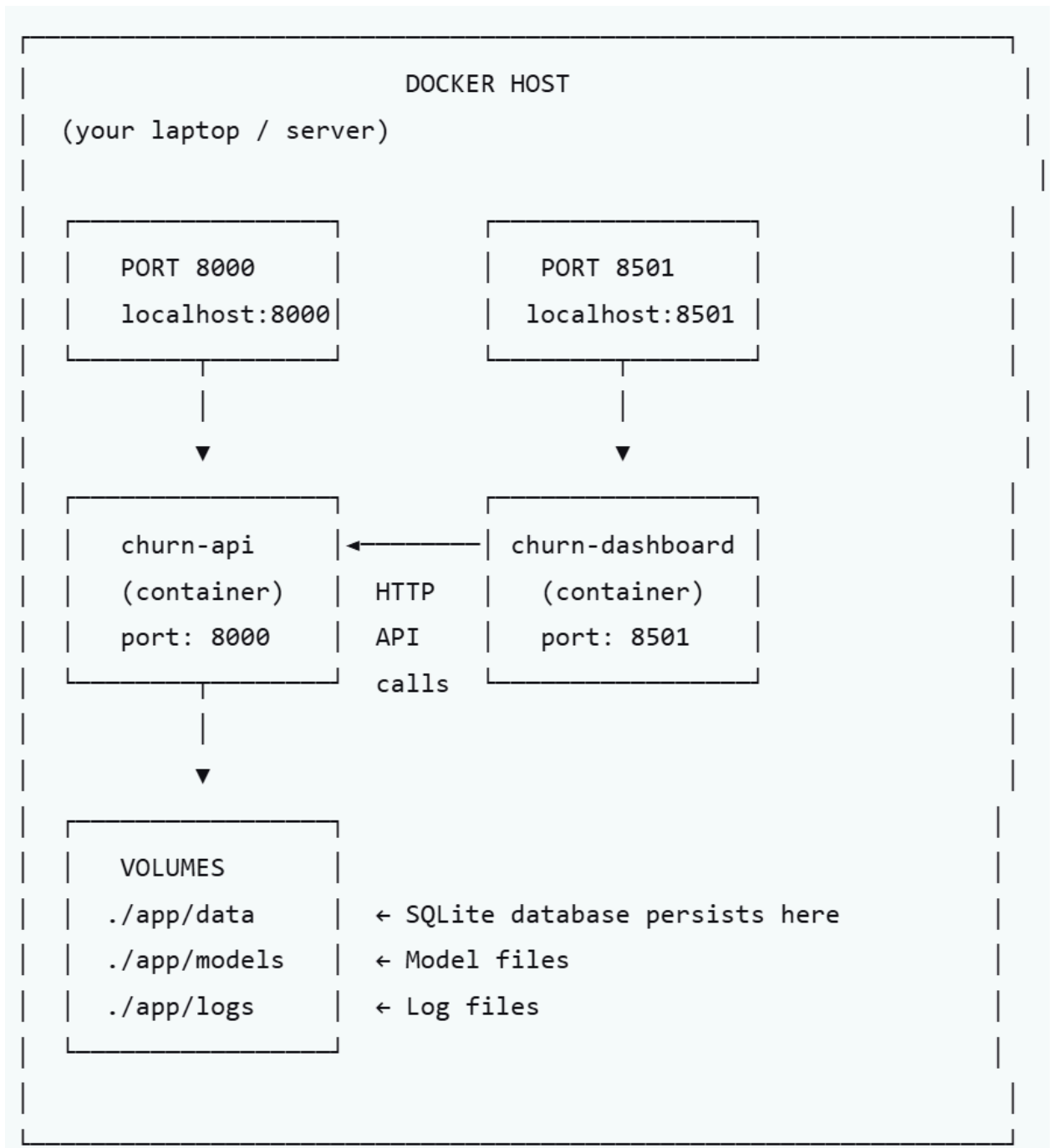
    ports:
      - "8501:8501"             # Access dashboard at Localhost:8501

    # Dashboard starts ONLY after API is ready
    depends_on:
      - api

    # Pass API URL as environment variable
    # 'api' resolves to the api service (Docker DNS)
    environment:
      - API_URL=http://api:8000

    restart: unless-stopped
```


Service Communication Diagram



Commands

Action	Command
Build and start all services	<code>docker-compose up --build</code>
Start in background (detached)	<code>docker-compose up -d</code>
Stop all services	<code>docker-compose down</code>
Stop and remove volumes (reset DB)	<code>docker-compose down -v</code>
View logs	<code>docker-compose logs -f</code>
Rebuild single service	<code>docker-compose build api</code>
Run commands in container	<code>docker exec -it churn-api bash</code>

Access Points After Deployment

Service	URL	Description
FastAPI	http://localhost:8000	API root (health check)
API Docs	http://localhost:8000/docs	Swagger UI (automatic)
Dashboard	http://localhost:8501	Streamlit monitoring UI

Volume Mounts Explained

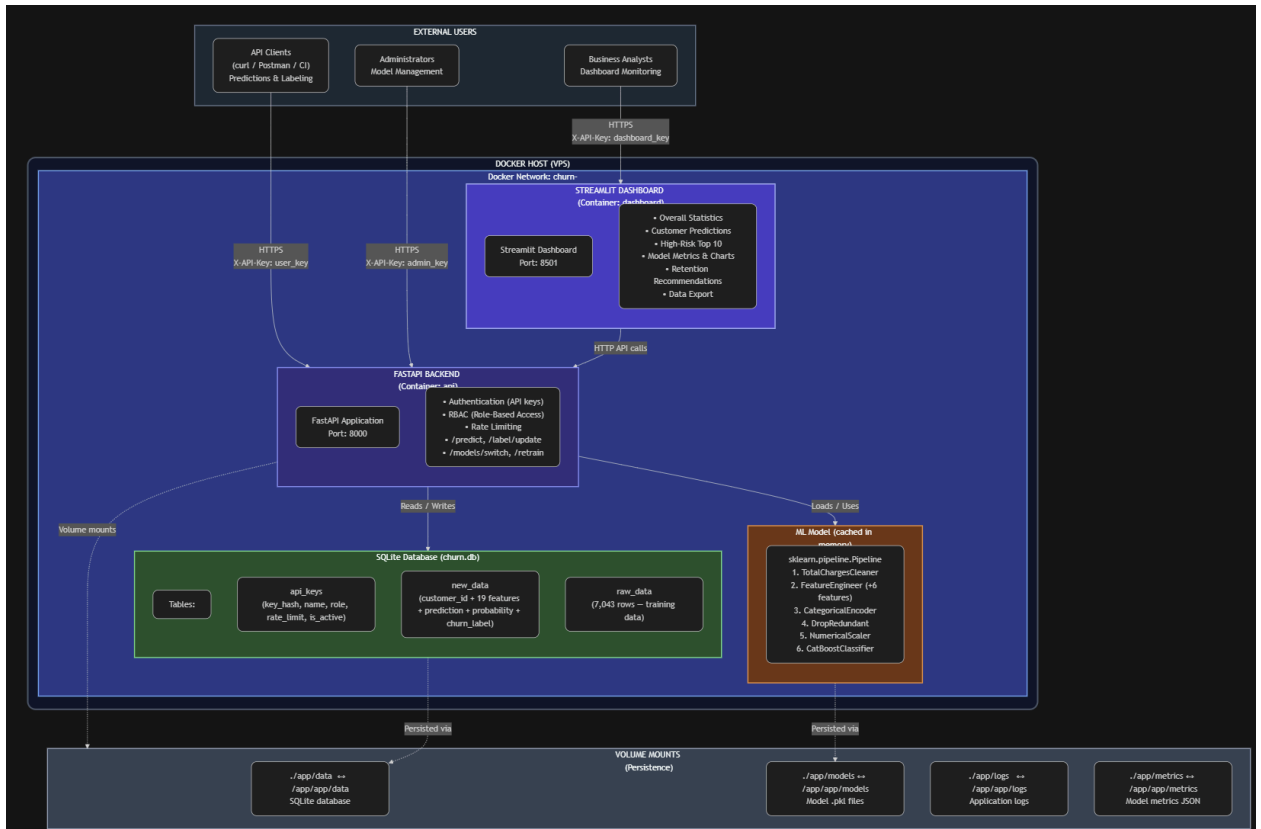
Host Path	Container Path	Purpose
<code>./app/data</code>	<code>/app/app/data</code>	SQLite <code>churn.db</code> - survives container restart
<code>./app/models</code>	<code>/app/app/models</code>	Trained <code>.pkl</code> model files
<code>./app/logs</code>	<code>/app/app/logs</code>	JSON logs for debugging
<code>./app/metrics</code>	<code>/app/app/metrics</code>	Model performance metrics

Why volumes? Without volumes, all data would be lost when container stops. With volumes, database and models persist across restarts.

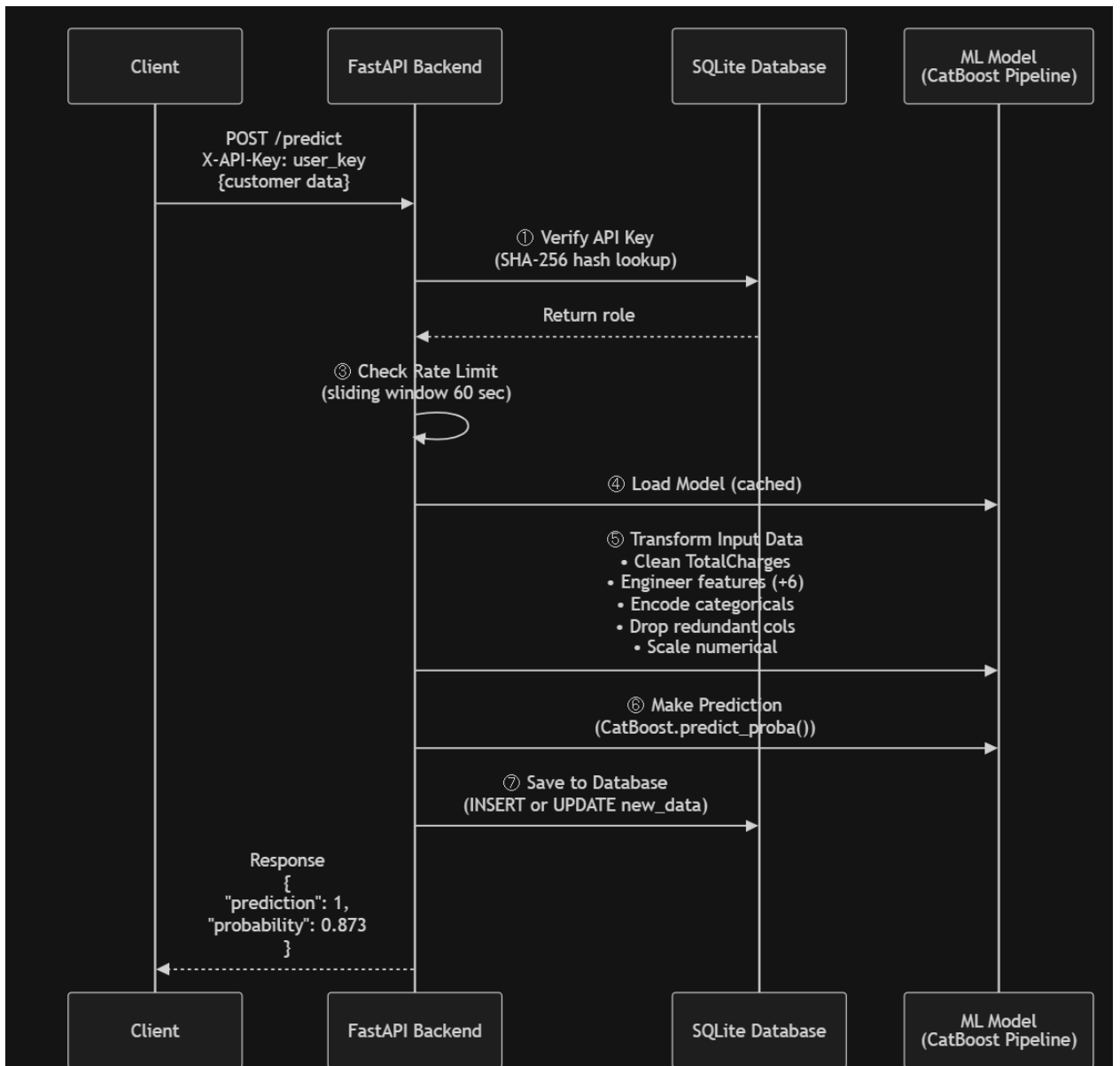
Part 11. PROJECT ARCHITECTURE

11.1. System Diagram

Component Interaction Diagram



Request Flow Diagram (API Prediction)



11.2. Technology Stack Component

Complete Technology Stack Table

Layer	Component	Technology	Version	Purpose
Backend Framework	FastAPI	FastAPI	0.115.12	REST API, automatic OpenAPI docs, async support
API Server	Uvicorn	Uvicorn	0.34.2	ASGI server for FastAPI
Data Validation	Pydantic	Pydantic	2.11.2	Request/response schema validation
Dashboard	Streamlit	Streamlit	1.37.1	Interactive monitoring dashboard
Visualization	Plotly	Plotly	5.24.1	Interactive charts (histograms, scatter, pie)
ML Framework	scikit-learn	scikit-learn	1.5.1	Pipeline, transformers, metrics
Gradient Boosting	CatBoost	CatBoost	1.2.10	Core ML model (classifier)
Model Serialization	cloudpickle	cloudpickle	3.0.0	Save/load ML models (supports lambdas)
Data Processing	Pandas	pandas	2.2.2	DataFrames, transformations
Numerical Computing	NumPy	numpy	1.26.4	Array operations, math functions
Database	SQLite	SQLite	3.x	Embedded database (no separate server)
HTTP Client	Requests	requests	2.32.3	Dashboard → API calls
Testing	pytest	pytest	7.4.4	Unit and integration tests
HTTP Testing	httpx	httpx	0.27.0	Async HTTP client for tests
Logging	Python logging	logging	built-in	JSON logs + rotation
Containerization	Docker	Docker	24.x	Container orchestration
Container Runtime	Docker Compose	Docker Compose	2.x	Multi-container management
Deployment	VPS	Ubuntu 22.04+	-	Production hosting

Backend Components (FastAPI)

Module	File	Responsibility
Main Application	<code>main.py</code>	Endpoint definitions, routing, startup event
Authentication	<code>auth.py</code>	API key verification, RBAC, key management
Rate Limiting	<code>rate_limit.py</code>	Sliding window rate limiting (60 sec)
Configuration	<code>config.py</code>	Environment variables, settings
Schemas	<code>schemas.py</code>	Pydantic models (request/response validation)
Database Core	<code>core/database.py</code>	SQLite connection, table creation
Key Input	<code>core/key_input.py</code>	API key prompting at startup

ML Pipeline Components (Transformers)

Transformer	File	What It Does
TotalChargesCleaner	transformers.py	Converts TotalCharges to numeric, fixes NaN, tenure=0 → 0
FeatureEngineer	transformers.py	Creates 6 derived features (avg_monthly_spend, tenure_group, has_internet, has_addons, risky_contract, auto_payment)
CategoricalEncoder	transformers.py	One-hot encoding for nominal, binary mapping for Yes/No
DropRedundant	transformers.py	Removes correlated/unnecessary columns
NumericalScaler	transformers.py	StandardScaler for numeric features

Scripts Organization

Script	Purpose	Called By
predict.py	Single prediction + DB save	POST /predict
predict_batch.py	Batch predictions + DB save	POST /predict/batch
put_lbl.py	Label management (CRUD)	POST /label/, GET /label/
model.py	Model loading (singleton pattern)	All prediction endpoints
model_manager.py	List, switch, delete models	GET /models, POST /models/switch
retrain.py	Incremental/full retraining	POST /retrain/*
evaluate_model.py	Test models on new data	POST /test/{model_name}
transformers.py	Custom sklearn transformers	Used in pipeline
init_keys.py	Manual API key setup	CLI script

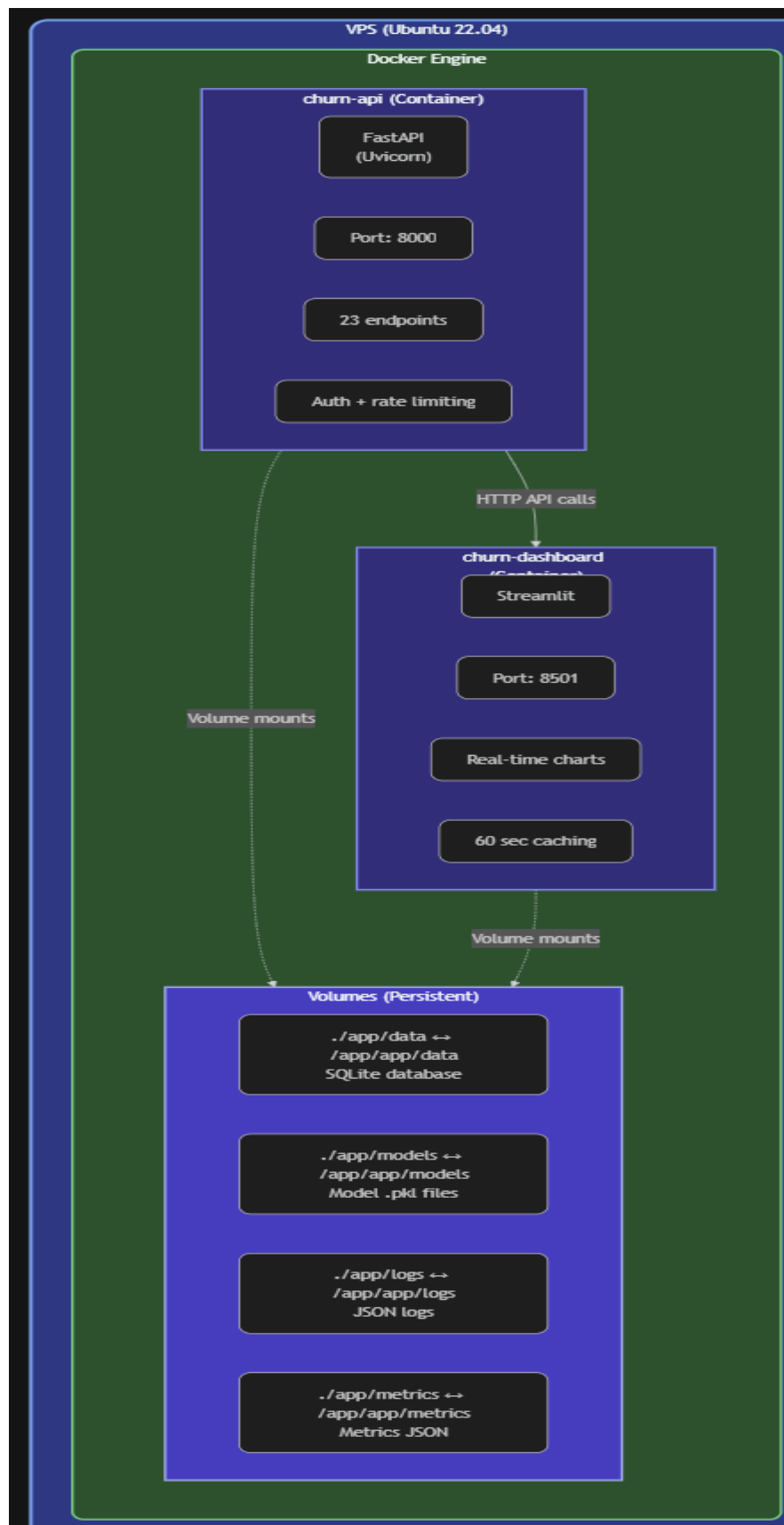
Port Mapping

Service	Container Port	Host Port	Access URL
FastAPI API	8000	8000	http://localhost:8000
FastAPI Docs	8000	8000	http://localhost:8000/docs
Streamlit Dashboard	8501	8501	http://localhost:8501

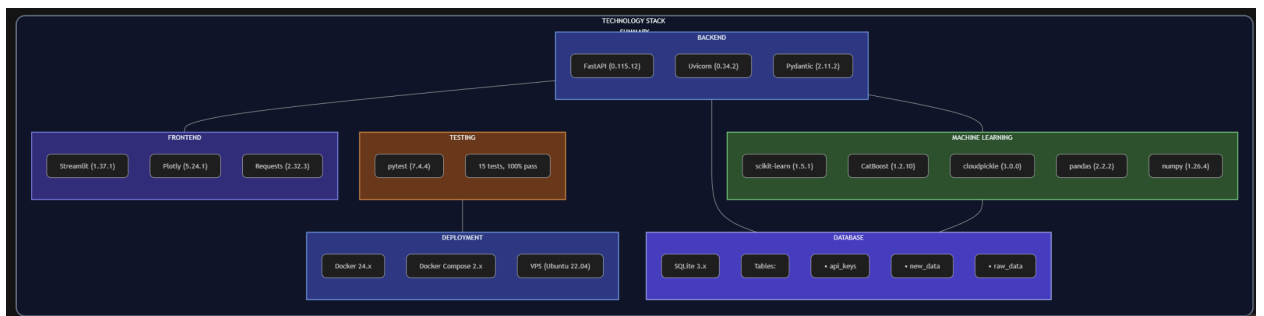
Technology Selection Rationale

Decision	Why Chosen
FastAPI	Fast, async, automatic OpenAPI docs, type hints, high performance
SQLite	Zero config, file-based, perfect for scale (up to 100k records), no separate server
Streamlit	Minimal code for dashboards, automatic refresh, built-in caching
CatBoost	Native categorical support, handles missing values, no manual encoding
cloudpickle	Serializes lambdas and custom classes (unlike standard pickle)
Docker	Reproducible environments, easy deployment, isolation

Deployment Architecture (Production)



Technology Stack Summary (Quick Reference)



Key Data Flow Paths

Operation	Path	Components Involved
Single Prediction	Client → FastAPI → Auth → Rate Limit → Model → Database → Client	auth.py, rate_limit.py, model.py, predict.py, database.py
Batch Prediction	Client → FastAPI → Auth → Rate Limit → Model → Database → Client	auth.py, rate_limit.py, model.py, predict_batch.py
Dashboard View	Browser → Streamlit → FastAPI → Database → Streamlit → Browser	call_api(), st.cache_data, dashboard.py
Label Update	Client → FastAPI → Auth → Rate Limit → Database → Client	auth.py, rate_limit.py, put_lbl.py
Incremental Retraining	Admin → FastAPI → Auth → Load Model → Transform → Continue Training → Save → Client	auth.py, retrain.py, model.py, transformers.py
Full Retraining	Admin → FastAPI → Auth → Load Old + New Data → Build Pipeline → Train → Save → Client	auth.py, retrain.py, transformers.py
Model Switch	Admin → FastAPI → Auth → Update Config → Clear Cache → Client	auth.py, model_manager.py, config.py

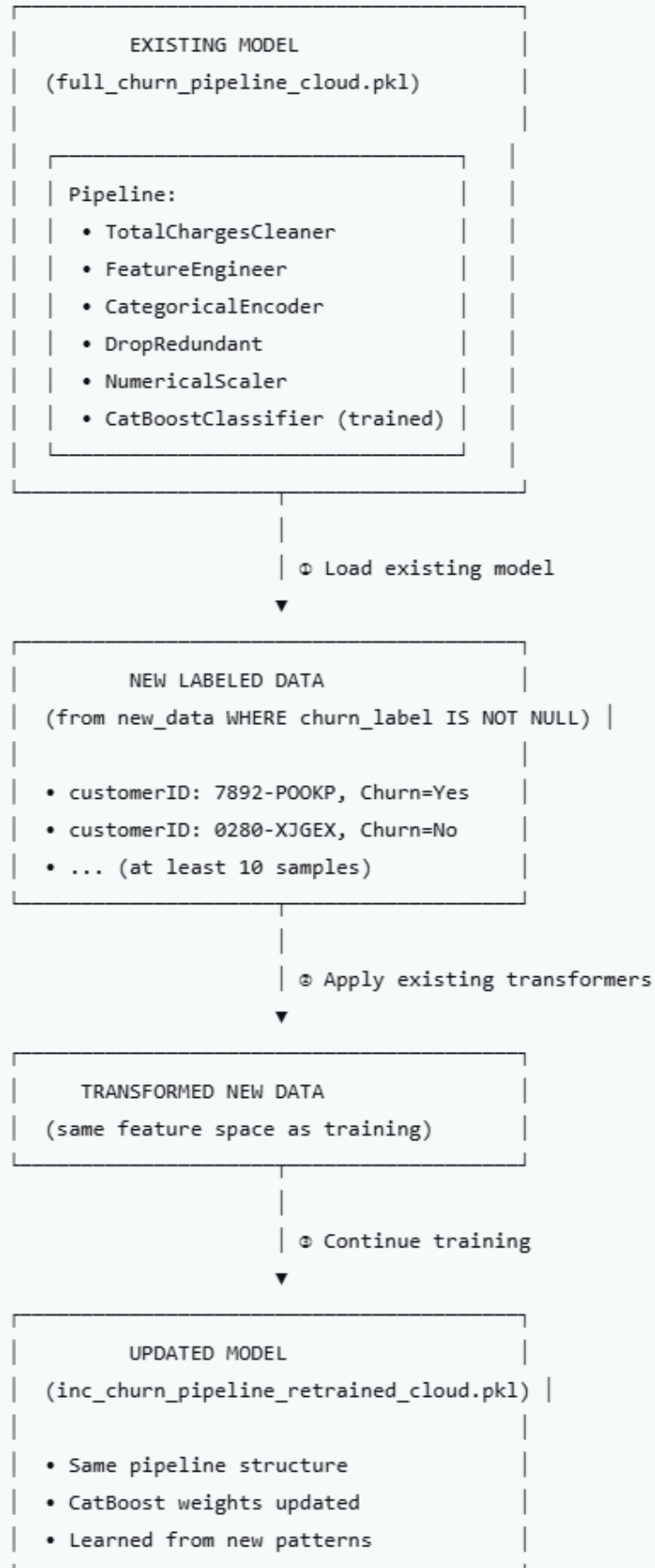
Part 12. RETRAINING

12.1. Incremental Retraining

Incremental retraining continues training the existing model on new labeled data without starting from scratch. This is the fastest and most efficient way to update the model with recent customer behavior.

How Incremental Retraining Works

INCREMENTAL RETRAINING FLOW



Key Characteristics

Aspect	Description
Speed	Fast (seconds to minutes, not hours)
Data required	Minimum 10 labeled samples (Yes + No)
Memory usage	Low (loads existing model + new data only)

Incremental vs Full Retraining Comparison

Aspect	Incremental	Full
Training time	Fast (minutes)	Slow (hours)
Data source	New labeled only	Old + new labeled
Model state	Continues from existing	Starts from scratch
Backup file	<code>inc*_backup.pkl</code>	<code>full*_backup.pkl</code>
Output file	<code>inc*_retrained.pkl</code>	<code>full*_retrained.pkl</code>

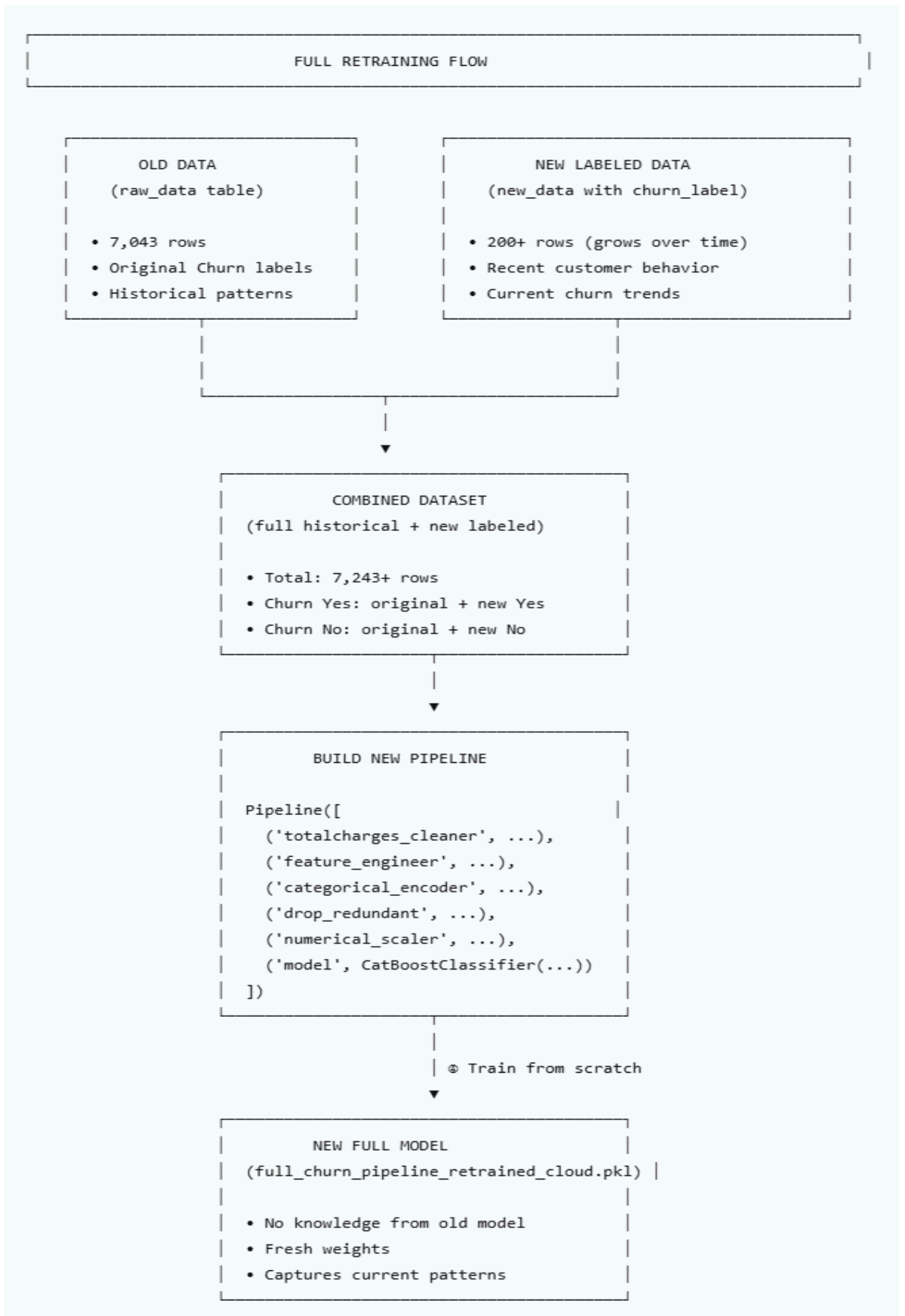
12.2. Full Retraining

Full retraining builds a completely new model from scratch using all available data (original training data + newly labeled data). This is slower but can capture fundamental changes in customer behavior.

When to Use Full Retraining

Scenario	Why Full Retraining
Concept drift	Customer behavior patterns have fundamentally changed
New features added	Model needs to incorporate additional data columns
Poor incremental results	Incremental model performance degraded
Quarterly refresh	Scheduled complete rebuild
Pipeline changes	Transformers or preprocessing modified

How Full Retraining Works



API Endpoints for Retraining

Endpoint	Method	Description	Required Role
/retrain/status	GET	Check if enough labeled data available	Admin
/retrain/incremental	POST	Incremental retraining on new data	Admin
/retrain/full	POST	Full retraining from scratch	Admin

Check retraining status:

```
curl -X GET "http://localhost:8000/retrain/status" \  
-H "X-API-Key: admin_key_here"
```

Response:

```
{  
  "status": "ready",  
  "total_records": 223,  
  "labeled_records": 202,  
  "churn_yes": 71,  
  "churn_no": 131,  
  "can_retrain": true,  
  "message": "Ready to retrain on 202 labeled samples"  
}
```

Start incremental retraining:

```
curl -X POST "http://localhost:8000/retrain/incremental" \  
-H "X-API-Key: admin_key_here"
```

Response:

```
{  
  "status": "success",  
  "message": "Model incrementally retrained on 202 labeled samples",  
  "samples_used": 202,  
  "churn_yes": 71,  
  "churn_no": 131,  
  "metrics": {  
    "accuracy": 0.7079,  
    "precision": 0.5556,  
    "recall": 0.8451,  
    "f1_score": 0.6704  
  },  
  "model_path": "app/models/inc_churn_pipeline_retrained_cloud.pkl",  
  "timestamp": "2026-04-20T14:30:00"  
}
```

To switch to retrained model: Use `POST /models/switch` with
`model_name=inc_churn_pipeline_retrained_cloud.pkl`

Part 13. CONCLUSION

13.1. Summary

This project successfully delivered a complete, production-ready customer churn prediction service for telecommunications companies. The solution combines a high-performance FastAPI backend, a CatBoost machine learning model with 95.7% recall, an interactive Streamlit dashboard, a manual labeling system for continuous data collection, and a fully automated retraining pipeline — all containerized with Docker and deployed on a live VPS server.

What was accomplished:

- Built and optimized a CatBoost model achieving recall = 95.72%, ROC-AUC = 0.8435, and F1-score = 0.5529 on the test set.
- Developed a full REST API with role-based authentication (user / admin / dashboard), rate limiting, and comprehensive Swagger documentation.
- Created an interactive Streamlit dashboard for real-time monitoring, churn risk visualization, monetary impact analysis, and manual labeling.
- Implemented a production-grade labeling system (CRUD + batch operations) with SQLite persistence.
- Designed a complete ML pipeline (custom scikit-learn transformers + CatBoost) that handles raw data → predictions in one call.
- Containerized the entire stack with Docker Compose and deployed it publicly with zero-downtime model switching capability.
- Added incremental and full retraining endpoints for continuous model improvement.

Results achieved:

- The model detects 96 out of 100 actual churners, enabling proactive retention campaigns.
- For a mid-size telecom (100,000 customers), the service can save ~\$4.97 million annually in revenue (based on 35% retention success rate).
- ROI exceeds 33,000% with payback period under 1 hour of operation.
- The system is fully operational, documented, tested, and ready for enterprise integration.

13.2. Future Improvements

The current solution is already production-grade, but the following enhancements are planned to further increase scalability, reliability, and business value:

- **Database upgrade** — Replace SQLite with PostgreSQL for better concurrency, ACID compliance, and horizontal scaling.
- **Monitoring & alerting** — Add Prometheus + Grafana for real-time metrics (API latency, prediction volume, model drift) and Slack/Telegram alerts.
- **A/B testing framework** — Enable simultaneous deployment of multiple model versions with traffic splitting to measure real-world retention impact.
- **Customer segmentation** — Extend the model with clustering (RFM + churn probability) for more personalized retention offers.

These improvements will evolve the service from a solid MVP into a full-scale enterprise MLOps platform.

Part 14. APPENDICES

Appendix A. Links

GitHub repository -

https://github.com/greencat1/Portfolio/tree/main/pet_projects/customer-churn-prediction

Live service - <http://207.231.105.113:8000/docs>

<http://207.231.105.113:8501/>